DAYANANDA SAGAR COLLEGE OF ENGINEERING

(An Autonomous Institute Affiliated to VTU, Belagavi, Accredited by NAAC with 'A' Grade)

Shavige Malleshwara Hills, Kumaraswamy Layout, Bengaluru-560111

Department of Artificial Intelligence & Machine Learning

DATA STRUCTURES WITH APPLICATIONS

LABORATORY MANUAL

III Semester

Course Code: 22AI34

Course Type: IPCC

Scheme: 2022-2026

Version 1.1

w.e.f. October,2023

Editorial Committee

Data Structures Laboratory Faculty, Dept. of AI&ML

Approved by

Dr. Vindhya P Malagi, H.O.D

Document Log

Name of the Document	Data Structures with Applications Laboratory Manual
Syllabus Scheme	2022-2026
Current Version and Date	Version 1.1, Oct 2023
Subject Code	22AI34
Editorial Committee	Data Structures with Applications Laboratory Faculty
Approved by	HOD, Dept. of A I & M L
Lab Faculty	Dr. Reshma S Dr. Archuda A Prof. Rashmi K
Lab Instructor	Miss. Chitrashree S

Table of Contents

	Program 4	Design, Develop and Implement a Program in C for converting an Infix Expression to Postfix Expression. Program should support for both parenthesized and free parenthesized expressions with the operators: **+, -, *, /, %(Remainder), ^(Power)** and **alphanumeric** operands**.**	**24**
	Program 5	Design, Develop and Implement a Program in C for the following Stack Applications a. Evaluation of Suffix expression with single digit operands and operators: +, -, *, /, %, ^ b. Solving Tower of Hanoi problem with n disks	**27**
	Program 6	Design, Develop and Implement a menu driven Program in C for the following operations on **Circular QUEUE** of Characters (Array Implementation of Queue with maximum size **MAX**) a. Insert an Element on to Circular QUEUE b. Delete an Element from Circular QUEUE c. Demonstrate ***Overflow*** and ***Underflow*** situations on Circular QUEUE d. Display the status of Circular QUEUE e. Exit Support the program with appropriate functions for each of the above operations.	**31**
	Program 7	Design, Develop and Implement a menu driven Program in C for the following operations on **Singly Linked List (SLL)** of Student Data with the fields: ***USN, Name, Branch, Sem, PhNo*** **a.** Create a **SLL** of **N** Students Data by using ***front insertion***. **b.** Display the status of **SLL** and count the number of nodes in it **c.** Perform Insertion / Deletion at End of **SLL** **d.** Perform Insertion /Deletion at Front of **SLL(Demonstration of stack)** **e.** Exit	**35**
	Program 8	Design, Develop and Implement a menu driven Program in C for the following operations on **Doubly Linked List (DLL)** of Employee Data with the fields: ***SSN, Name, Dept, Designation,*** ***Sal, PhNo.*** **a.** Create a **DLL** of **N** Employees Data by using ***end insertion***. **b.** Display the status of **DLL** and count the number of nodes in it **c.** Perform Insertion and Deletion at End of **DLL** **d.** Perform Insertion and Deletion at Front of **DLL** **e.** Demonstrate how this **DLL** can be used as **Double Ended Queue** **f.** Exit	**44**

Vision of the Department

To provide progressive education and flourish the student's ingenuity to be successful professionals impacting the society for a smarter and ethical world.

Mission of the Department

M1: *To adopt an engaging teaching learning process with emphasis on problem solving and programming skills.*

M2: *To promote additional skill development through enhanced and experiential learning.*

M3: *To collaborate with industries and professional bodies and make the students industry ready.*

M4: *To encourage innovation through multi-disciplinary research and development activities.*

M5: *To imbibe human values and ethics in students to make them impact the society's as responsible professionals.*

Program Educational Objectives (PEOs) of Department

After the course completion, AI&ML graduates will be able to:

PEO1: To practice and implement their success skills of problem solving, communication. and collaboration for providing innovative engineering solutions.

PEO2: Contribute their AI & ML expertise grounded in computer science and mathematics as members and leaders of professional engineering teams in multidisciplinary domains.

PEO3: Demonstrate lifelong learning through continued professional development and higher education in top graduate technical, research and management programs.

PEO4: Demonstrate a commitment to society by applying the skills and knowledge for a smarter and ethical world.

Program Specific Outcomes (PSOs)

PSO1: Graduates will be adept in programming and problem-solving skills for developing and managing software.

PSO2: Graduates will be able to identify, formulate , predict , and solve real world Problems by applying principles of Artificial Intelligence & Machine Learning.

PSO3: Graduates will be proficient to efficiently manage computer system resources. using cloud computing technologies.

Program Outcomes (POs)

Engineering Graduates will be able to:

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

2. **Problem analysis**: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

3. **Design/development of solutions**: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

4. **Conduct investigations of complex problems**: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

5. **Modern tool usage**: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

6. **The engineer and society**: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

7. **Environment and sustainability**: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

8. **Ethics**: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

9. **Individual and team work**: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

10. **Communication**: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

11. **Project management and finance**: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

12. **Life-long learning**: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

SYLLABUS

Course code: 22AI34 Credits: 04 L: P: T: S: 0:2:2:0

CIE Marks: 50 Exam Hours: 03

SEE Marks: 50 Total Hours: 52

CREDIT-2

Course Name:DATA STRUCTURES WITH APPLICATIONS LABORATORY

Course prerequisite: C Programming for Problem Solving

Course Learning Objectives:

This course (22AI34) will enable students to get practical experience in design, develop, implement, analyze and evaluation/testing of

- Introduce the concept of data structures through ADT.
- Linear data structures and their applications such as stacks, queues, and lists
- Non-Linear data structures and their applications such as trees and graphs
- To develop application using data structure algorithms

Implement all the programs in 'C / C++' Programming Language and Linux /Windows

1. Design, Develop and Implement a menu driven Program in C for the following Array operations.
   f. Creating an Array of **N** Integer Elements
   g. Display of Array Elements with Suitable Headings
   h. Inserting an Element (**ELEM**) at a given valid Position (**POS**)
   i. Deleting an Element at a given valid Position (**POS**)
   j. Exit.

Support the program with functions for each of the above operations.

2. Design, Develop and Implement a Program in C for the following operations on **Strings**
   c. Read a main String (**STR),** a Pattern String (**PAT**) and a Replace String (**REP**)
   d. Perform Pattern Matching Operation: Find and Replace all occurrences of **PAT** in **STR** with **REP** if **PAT** exists in **STR.** Report suitable messages in case **PAT** does not exist in **STR**

Support the program with functions for each of the above operations.

3. Design, Develop and Implement a menu driven Program in C for the following operations on **STACK** of Integers (Array Implementation of Stack with maximum size **MAX**)
   g. ***Push*** an Element on to Stack
   h. ***Pop*** an Element from Stack
   i. Demonstrate how Stack can be used to check ***Palindrome***
   j. Demonstrate ***Overflow*** and ***Underflow*** situations on Stack
   k. Display the status of Stack
   l. Exit

Support the program with appropriate functions for each of the above operations.

4. Design, Develop and Implement a Program in C for converting an Infix Expression to Postfix

Expression. Program should support for both parenthesized and free parenthesized expressions with the operators: **+, -, *, /, %(Remainder), ^(Power)** and **alphanumeric** operands**.**

5. Design, Develop and Implement a Program in C for the following Stack Applications

a. Evaluation of **Suffix expression** with single digit operands and operators: **+, -, *, /,%, ^**

b. Solving **Tower of Hanoi** problem with **n** disks

6. Design, Develop and Implement a menu driven Program in C for the following operations on **Circular QUEUE** of Characters (Array Implementation of Queue with maximum size **MAX**)

f. Insert an Element on to Circular QUEUE
g. Delete an Element from Circular QUEUE
h. Demonstrate ***Overflow*** and ***Underflow*** situations on Circular QUEUE
i. Display the status of Circular QUEUE
j. Exit

Support the program with appropriate functions for each of the above operations.

7. Design, Develop and Implement a menu driven Program in C for the following operations on **Singly Linked List (SLL)** of Student Data with the fields: ***USN, Name, Branch, Sem, PhNo***

f. Create a **SLL** of **N** Students Data by using ***front insertion***.
g. Display the status of **SLL** and count the number of nodes in it
h. Perform Insertion / Deletion at End of **SLL**
i. Perform Insertion /Deletion at Front of **SLL(Demonstration of stack)**
j. Exit

8. Design, Develop and Implement a menu driven Program in C for the following operations on **Doubly Linked List (DLL)** of Employee Data with the fields: ***SSN, Name, Dept, Designation, Sal, PhNo.***

g. Create a **DLL** of **N** Employees Data by using ***end insertion***.
h. Display the status of **DLL** and count the number of nodes in it
i. Perform Insertion and Deletion at End of **DLL**
j. Perform Insertion and Deletion at Front of **DLL**
k. Demonstrate how this **DLL** can be used as **Double Ended Queue**
l. Exit

9. Design, Develop and Implement a Program in C for the following operations on **Singly Circular Linked List (SCLL)** with header nodes

c. Represent and Evaluate a Polynomial P(x,y,z) = $\mathbf{6x^2y^2z-4yz^5+3x^3yz+2xy^5z-2xyz^3}$

d. Find the sum of two polynomials POLY1(x,y,z) and POLY2(x,y,z) and store the result in POLYSUM(x,y,z)

Support the program with appropriate functions for each of the above operations.

10. Design, Develop and Implement a menu driven Program in C for the following operations on **Binary Search Tree (BST)** of Integers

e. Create a BST of **N** Integers: 6, 9, 5, 2, 8, 15, 24, 14, 7, 8, 5, 2
f. Traverse the BST in Inorder, Preorder and Post Order
g. Search the BST for a given element (**KEY**) and report the appropriate message.
h. Exit

11. Design, Develop and Implement a Program in C for the following operations on **Graph (G)** of Cities

c. Create a Graph of N cities using Adjacency Matrix.
d. Print all the nodes reachable from a given starting node in a digraph using BFS/DFS method

12. Given a File of **N** employee records with a set **K** of Keys(4-digit) which uniquely determine the records in file **F**. Assume that file **F** is maintained in memory by a Hash Table(HT) of **m**memory locations with **L** as the set of memory addresses (2-digit) of locations in HT. Let the keys in **K** and addresses in **L** are Integers. Design and develop a Program in C that uses Hash function **H: K->L** as H(**K**)=**K** mod **m (remainder** method), and implement hashing technique to map a given key **K** to the address space **L.** Resolve the collision (if any) using **linear probing**.

Course Outcomes

Upon successful completion of this course, students are able to

COs	Course Outcomes
22AI34.1	**Understand** the fundamentals of data structures and their significance in problem-solving using various programming techniques.
22AI34.2	**Apply** Linear data structures to solve engineering problems effectively.
22AI34.3	**Demonstrate** graph representations and hashing techniques.
22AI34.4	**Perform** various operations on tree, AVL tree & RB tree.
22AI34.5	**Analyse** the operations on various Data Structures and demonstrate their applications.
22AI34.6	**Illustrate** tree traversal methods to manage engineering data effectively.

CO-PO-PSO MAPPING

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
22AI34.1	**3**											**3**
22AI34.2	**3**				**2**							**3**
22AI34.3	**3**	**3**			**2**							**3**
22AI34.4	**3**	**3**			**2**							**3**
22AI34.5	**3**	**3**			**2**				**3**	**3**		**3**
22AI34.6	**3**	**3**			**2**				**3**	**3**		**3**

LAB EVALUATION PROCESS

CIE-Practical (Conduction) :30 Min 12

WEEK WISE EVALUATION OF EACH PROGRAM

ACTIVITY	MARKS
Execution	20
Observation Book + Record	10
TOTAL	30

CIE-Practical (Test) :20 Min 08

INTERNAL ASSESSMENT EVALUATION (End of Semester)		
S.No	**ACTIVITY**	**MARKS**
01	Write-up	10
02	Conduction	35
03	Viva Voce	05
	Total	50 (Reduced to 20 min 08)

Total CIE-Practical :50 Min 20 (Reduced to 20 Min 08)

FINAL INTERNAL ASSESSMENT CALCULATION		
S.No	**ACTIVITY**	**MARKS**
01	Average of weekly entries	30
02	Internal Assessment reduced to	20
	Total	50

Reduce overall lab marks from 50 to 20. The minimum marks to be secured in CIE of the LAB to appear for SEE of IPCC shall be the 8 marks [40% of maximum marks (20)].

CIE-Practical (CONTINUOUS INTERNAL EVALUATION) : 30

Continuous Evaluation done every week.

Rubrics	MARKS
Observation	5
Record	5
Conduction	10
Viva	5
Attendance	5
TOTAL	**30**

Rubrics for Continuous Internal Evaluation :

Table 1: Rubrics for lab CIE Max Marks: 20 (excluding attendance)

Evaluation / Rubrics	Not Satisfactory (1-2)	Satisfactory (3)	Good (4)	Excellent (5)
Observation (Inferring to the programs in the lab & documenting) (05 M)	Incomplete programs and missing analysis part, late submission	Complete programs with comments however analysis and inference not presented.	Complete programs with comments including analysis, inference not. Completed on time	Complete programs with comments including analysis and inference provided and completed on time
Conduction (5*2 M)	Executed the program without understanding, rectified the errors andlack of conceptual understanding.	Successfully executed the program with basic understanding of the program	Successfully executed the program with good understanding of the program (function level)	Successfully executed the program with detailed understanding of the program (line by line)
Records (05 M)	Incomplete programs and missing analysis part, late submission	Complete programs with comments however analysis and inference not presented.	Complete programs with comments including analysis, however inference not presented appropriately. Submitted on time.	Complete programs with comments including analysis and inference provided & submitted on time
Viva (05 M)	1 question out of 5 viva questions answered on a one-on-one basis	Basic understanding on the programs and 2 or 3 questions out of 5 viva questions answered on a one-on-one basis	Good understanding on the programs and 4 questions out of 5 viva questions answered on a one-on-one basis	Programs well understood and all 5 questions answered on a one-on-one basis
Score	Score 1 (S1)	Score 2 (S2)	Score 3 (S3)	Score 4 (S4)
Total Score				**S1+ S2+S3+S4**

In the conduction criteria of CIE (During internal lab exams), the following Rubrics are considered.

- Syntax
- Logic
- Correctness
- Completeness of the program

Table 2: Rubrics for lab programs conduction component Max Marks:5

Rubrics	**Not Satisfactory (1-2)**	**Satisfactory (3)**	**Good (4)**	**Excellent (5)**
Syntax (5) Ability to understand and follow the rules of the program	Program doesn't compile and has errors and warnings	Program compiles, but contains warnings that lead to incorrect logic	Program compiles and it is free from errors but may contain non-standard usage of programming	Program compiles without errors and follows the standards of programming
Logic (5) Ability to identify conditions, control flow and data structures for the given program	Incorrect program logic such as infinite loops	Program logic is correct without any infinite loops, however all possible boundary cases not addressed	Program logic is correct with few boundary conditions	Program logic is correct with all conditions and cases coded with no ambiguity
Correctness (5) Ability to code algorithms that produce correct appropriate output for the given program	Incorrect output or inappropriate output for any given test case	Program gives correct output for most inputs but not for all test cases	Program gives correct output for most test cases	Program gives correct output for all test cases
Completeness (5) Ability to apply critical analysis to check for the completeness of the program	Program shows little or no recognition on different cases leading to incompleteness	Program shows evidence of case analysis and missing significant cases or missed cases	Most of the requirements of the program met however all possible cases not handled	All requirements of the program met and all possible cases handled
Total				**Score /4**

Rubrics for write up and execution of Internal Examination

TOPIC	MARKS
Write up	10
Execution	35
Viva	5
TOTAL	50

Table 3: Rubrics for write up Internal Examination

Max Marks:10

	Rubrics	Not Satisfactory (1-2)	Satisfactory (3-5)	Good (6-8)	Excellent (9-10)
WRITE-UP	**Syntax (5)** Ability to understand and follow the rules of the program	Program doesn't compile and has errors and warnings	Program compiles, but contains warnings that lead to incorrect logic	Program compiles and it is free from errors but may contain non-standard usage of programming	Program compiles without errors and follows the standards of programming
	Logic (5) Ability to identify conditions, control flow and data structures for the given program	Incorrect program logic such as infinite loops	Program logic is correct without any infinite loops, however all possible boundary cases not addressed	Program logic is correct with few boundary conditions	Program logic is correct with all conditions and cases coded with no ambiguity
	Total				**Score*1**

Table 4: Rubrics for execution of Internal Examination

Max Marks:35

	Rubrics	Not Satisfactory (1-2)	Satisfactory (3)	Good (4)	Excellent (5)
EXECUTION	**Correctness (5)** Ability to code algorithms that produce correct appropriate output for the given program	Incorrect output or inappropriate output for any given test case	Program gives correct output for most inputs but not for all test cases	Program gives correct output for most test cases	Program gives correct output for all test cases
	Completeness (5) Ability to apply critical analysis to check for the completeness of the program	Program shows little or no recognition on different cases leading to incompleteness	Program shows evidence of case analysis and missing significant cases or missed cases	Most of the requirements of the program met however all possible cases not handled	All requirements of the program met and all possible cases handled
	Total				**Score*3.5**

CHAPTER 1

INTRODUCTION TO DATA STRUCTURES

Data Structure is defined as the way in which data is organized in the memorylocation. There are 2 types of datastructures:

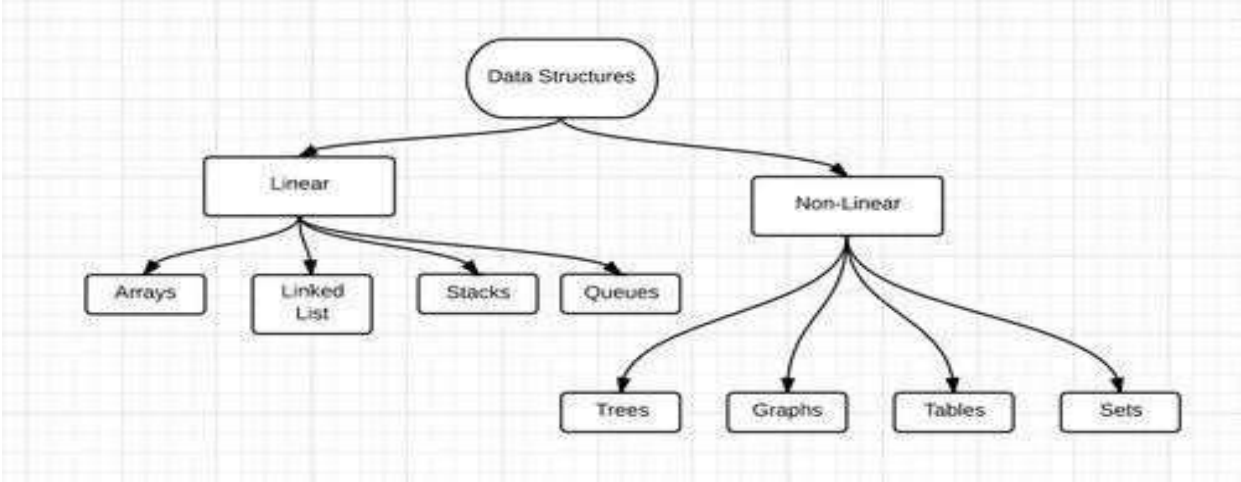


Fig 1: Classification of Data Structures

Linear Data Structure:

In linear data structure all the data are stored linearly or contiguously in the memory. All the data are saved in continuously memory locations and hence all data elements are saved in one boundary. A linear data structure is one in which we can reach directly only one element from another while travelling sequentially. The main advantage, we find the first element, and then it's easy to find the next data elements. The disadvantage, the size of the array must be known before allocating the memory.

The different types of linear data structures are:

- Array
- Stack
- Queue
- Linked List

Non-Linear Data Structure:

Every data item is attached to several other data items in a way that is specific for reflecting relationships. The data items are not arranged in a sequential structure.

The various types of non linear data structures are:

- Trees
- Graphs

CHAPTER 2

PROGRAM 1

Design, Develop and Implement a menu driven Program in C for the following Array operations

a. **Creating an Array of N Integer Elements**
b. **Display of Array Elements with Suitable Headings**
c. **Inserting an Element (ELEM) at a given valid Position (POS)**
d. **Deleting an Element at a given valid Position (POS)**
e. **Exit.**

Support the program with functions for each of the above operations.

ABOUT THE EXPERIMENT:

Arrays play a significant role in any programming language, as they allow the programmer to store more than one value in a variable. Array is a data structure consisting of a collection of elements of same datatype. This collection is arranged in a way such that each element can be uniquely identified and addressed. These addresses are called indices. These are integer values. The memory allocation foran array is done contiguously.

Why we need Array ?

Ex: inta[4]={20,30,5,10} and this array is represented as below.

Value/elements	20	30	5	10
index	1	2	3	5

Consider a scenario where you need to find out the average of 100 integer numbers entered by user. In C, you have two ways to do this:

1) Define 100 variables with int data type and then perform 100 scanf() operations to store the entered values in the variables and then at last calculate the average of them.

2) Have a single integer array to store all the values, loop the array to store all the entered values in array and later calculate the average.

Which solution is better according to you? Obviously the second solution, it is convenient to store same data types in one single variable and later access them using array.

OBJECTIVES:

- To learn how to use arrays for storing integer data.
- Perform Insertion and deletion operation by checking the valid position

In this Program the array can be represented as one / single dimensional elements. Menu driven program in C-language to perform various array operations are implemented with the help of user defined functions as following

a. create()
b. display()
c. insert()
d. delet()
e. exit()

REAL TIME APPLICATIONS:

- Arrays are used to implement mathematical vectors and matrices, as well as other kinds of rectangular tables
- Arrays are used to implement other data structures, such as lists, heaps, hash, tables, deques, queues, stacks, strings
- Arrangement of leader-board of a game can be done simply through arrays
- A simple question Paper is an array of numbered questions with each of them assigned to some marks.
- 2-D arrays, commonly known as, matrix, are used in image processing. It is also used in speech processing, in which each speech signal is an array.

Original array

Value/elements	20	30	5	10
index	1	2	3	5

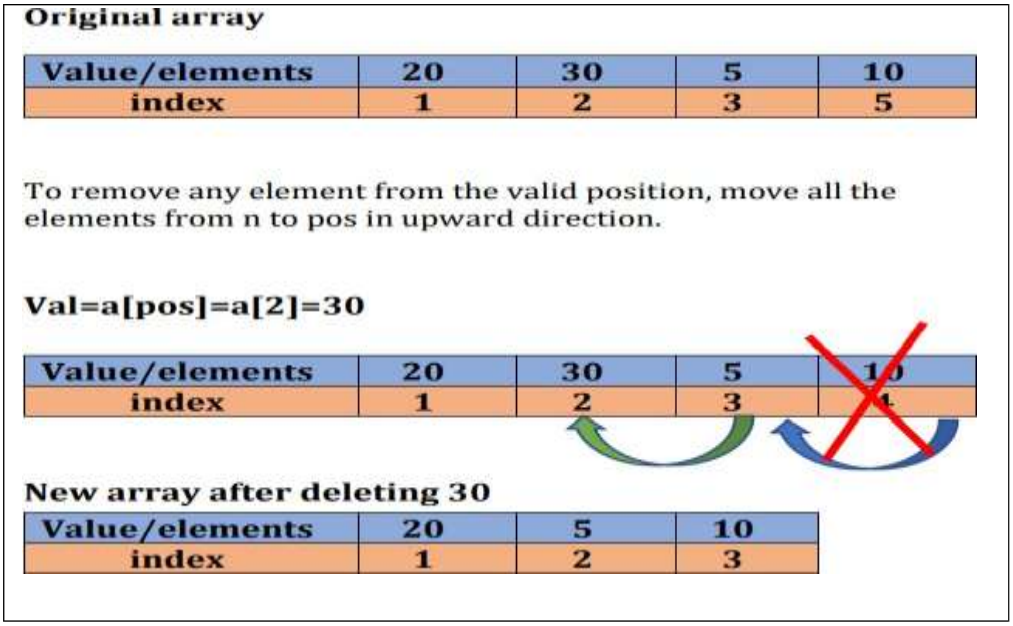
To insert any element to the valid position, move all the elements from pos to n in downward direction.

Pos: 2, Element to be inserted: 60.

New Array

60

Value/elements	20	60	30	5	10
index	1	2	3	4	5



Source Code:

```
#include<stdio.h>
#include<stdlib.h>
int a[20];
int n, val, i, pos, choice;
/*Function Prototype*/
void create();
void display();
void insert();
voiddelet();
int main()
{
        while(1)
        {
        printf("\n\n--------MENU ----------\n");
        printf("1.CREATE\n");
        printf("2.DISPLAY\n");
        printf("3.INSERT\n");
        printf("4.DELETE\n");
        printf("5.EXIT\n");
        printf(" ---------------------- ");
        printf("\nENTER YOUR CHOICE:\t");
        scanf("%d",&choice); //Reading user choice
        switch(choice)
           {
                case 1: create();          //to create an array
                        break;
                case 2: if(n)
                           display();  //to display the elements
                        else
```

```
                        {
                                printf("Array is empty\n");
                        }
                    break;

            case 3: :insert();  // to insert elements
                    break;

            case 4:if(n)
                      delet(); //to delete an item at specific position
                   else
                   {
                     printf("Array is empty\n");
                    }
                    break;
            case 5: exit(0);          //exit the program
                    break;
            default: printf("\nInvalid choice:\n");
                    break;
        }}
  return 0;
}
  //creating an array
  void create()
  {
        printf("\nEnter the size of the array elements:\t");
        scanf("%d",&n);
        printf("\nEnter the elements for the array:\n");
        for(i=1;i<=n;i++)
        {
          scanf("%d",&a[i]);
        }
  }
  //displaying an array elements
  void display()
  {
        inti;
        printf("\nThe array elements are:\n");
        for(i=1;i<=n;i++)
        {
          printf("%d\t",a[i]);
        }
  }
  //inserting an element into an array
  void insert()
  {
        printf("\nEnter the position for the new element:\t");
        scanf("%d",&pos);
        if(pos>(n+1))
```

```
        printf("\n Invalid Position\n");
        else
        {
                printf("\nEnter the element to be inserted :\t");
                scanf("%d",&val);
                for(i=n;i>=pos;i--)
                {
                        a[i+1]=a[i];
                }
                a[pos]=val;
                n=n+1;
        }
}
//deleting an array element
voiddelet()
{
        printf("\nEnter the position of the element to be deleted:\t");
        scanf("%d",&pos);
        if(pos>n)
        printf("Invalid Position\n");
        else
         {
                val=a[pos];
                for(i=pos;i<n;i++)
                {
                        a[i]=a[i+1];
                }
                n=n-1;
                printf("\nThe deleted element is =%d",val);
        }
}
```

EXPECTED OUTPUT:

```
--------MENU----------
1.CREATE
2.DISPLAY
3.INSERT
4.DELETE
5.EXIT
----------------------
ENTER YOUR CHOICE:      1

Enter the size of the array elements:   4

Enter the elements for the array:
20 5 10 30


--------MENU----------
1.CREATE
2.DISPLAY
3.INSERT
4.DELETE
5.EXIT
----------------------
ENTER YOUR CHOICE:      2

The array elements are:
20      5       10      30

--------MENU----------
1.CREATE
2.DISPLAY
3.INSERT
4.DELETE
5.EXIT
----------------------
ENTER YOUR CHOICE:      3

Enter the position for the new element: 1

Enter the element to be inserted :      90

--------MENU----------
1.CREATE
2.DISPLAY
3.INSERT
4.DELETE
5.EXIT
----------------------
ENTER YOUR CHOICE:      2

The array elements are:
90      20      5       10      30
```

```
--------MENU----------
1.CREATE
2.DISPLAY
3.INSERT
4.DELETE
5.EXIT
----------------------
ENTER YOUR CHOICE:      3

Enter the position for the new element: 9

 Invalid Position


--------MENU----------
1.CREATE
2.DISPLAY
3.INSERT
4.DELETE
5.EXIT
----------------------
ENTER YOUR CHOICE:      3

Enter the position for the new element: 6

Enter the element to be inserted :      60


--------MENU----------
1.CREATE
2.DISPLAY
3.INSERT
4.DELETE
5.EXIT
----------------------
ENTER YOUR CHOICE:      2

The array elements are:
90      20      5       10      30      60
ENTER YOUR CHOICE:      4

Enter the position of the element to be deleted:        1

The deleted elenent is =90

--------MENU----------
1.CREATE
2.DISPLAY
3.INSERT
4.DELETE
5.EXIT
----------------------
ENTER YOUR CHOICE:      4

Enter the position of the element to be deleted:        3

The deleted elenent is =10

--------MENU----------
1.CREATE
2.DISPLAY
3.INSERT
4.DELETE
5.EXIT
----------------------
ENTER YOUR CHOICE:      4

Enter the position of the element to be deleted:        8
Invalid Position
```

PROGRAM 2

Design, develop and implement a Program in C for the following operations on Strings

a. **Read a main String (STR), a Pattern String (PAT) and a Replace String (REP)**
b. **Perform Pattern Matching Operation: Find and Replace all occurrences of PAT in STR with REP if PAT exists in STR. Report suitable messages in case PAT does not exist in STR**

Support the program with functions for each of the above operations. Don't use Built-in functions.

ABOUT THE EXPERIMENT:

Strings are actually one-dimensional array of characters terminated by a null character '\0'. Thus a null-terminated string contains the characters followed by a null. The following declaration and initialization create a string consisting of the word "Hello". To hold the null character at the end of the array, the size of the character array containing the string is one more than the number of characters in the word "Hello."

char greeting[6] = {'H', 'e', 'l', 'l', 'o', '\0'};

If you follow the rule of array initialization then you can write the above statement as follows:

char greeting[ ] = "Hello";

C language supports a wide range of built-in functions that manipulate null-terminated strings as follows:

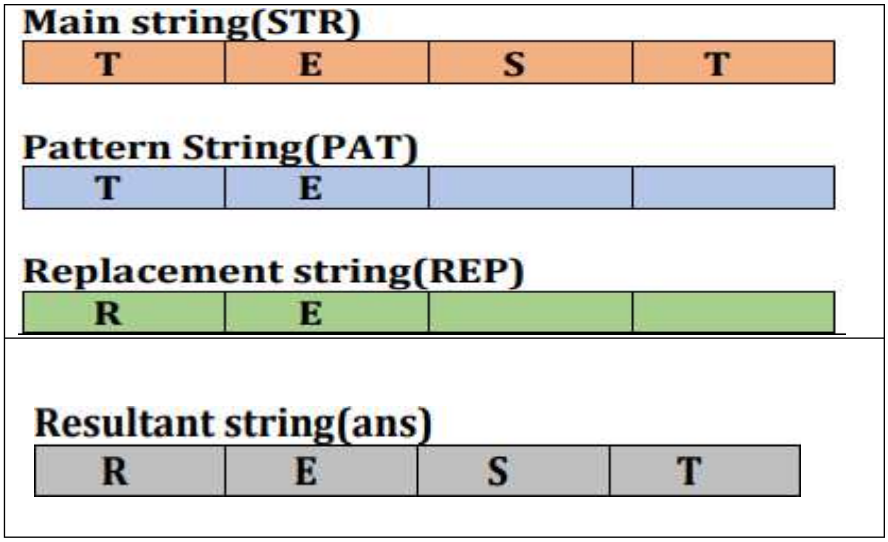
strcpy(s1, s2); Copies string s2 into string s1.
strcat(s1, s2); Concatenates string s2 onto the end of string s1.
strlen(s1); Returns the length of string s1.
strcmp(s1, s2); Returns 0 if s1 and s2 are the same; less than 0 if s1s2.
strchr(s1, ch); Returns a pointer to the first occurrence of character ch in string s1.
strstr(s1, s2); Returns a pointer to the first occurrence of string s2 in string s1

OBJECTIVES:

- Creation of main String (STR), a Pattern String (PAT) and a Replace String (REP)
- Finding PAT in STR
- if PAT is present in STR then replace all the occurrences of the PAT in STR by REP
- if PAT is not present then display “pattern not found”
- Perform this operations with the help of user-defined function read input () and patternmatch

REAL TIME APPLICATIONS:

- A few of its imperative applications are Spell Checkers, Spam Filters, Intrusion Detection System, Search Engines, Plagiarism Detection, Bioinformatics and DNA Sequencing, Digital Forensics and Information Retrieval Systems etc
- String matching is also used in the Database schema, Network systems.



Source Code

```
#include<stdio.h>
int flag=0;
void readinput(char str[],char pat[],char rep[])
{
        printf("Enter the Main String\n");
        gets(str);
        printf("Enter the Pattern String\n");
        gets(pat);
        printf("Enter the Replacement String\n");
        gets(rep);
}

void patternmatch(char str[],char pat[],char rep[], char ans[])
{
        int i,j,c,m,k;
        i=j=c=m=k=0;
        while(str[c] != '\0')
        {
                if(str[m] == pat[i])
                {
                        i++;m++;
                        if(pat[i]=='\0')
                        {
                                flag=1;
                                for(k=0;rep[k] != '\0';k++,j++)
                                ans[j] = rep[k];
                                i=0;
                                c=m;
                        }
                }
                else
                {
```

```
                        ans[j] = str[c];
                        j++;c++;
                        m=c;
                        i=0;
                }
        }
        ans[j]='\0';
}
void main()
{
        char str[100],pat[100],rep[100],ans[100];
        readinput(str,pat,rep);
        patternmatch(str,pat,rep,ans);
        if(flag == 0)
        {
                printf("Pattern Not Found\n");
                printf("Resultant String is: %s\n", ans);
        }
        else

        printf("Resultant String is: %s\n", ans);
}
```

EXPECTED OUTPUT:

```
Enter the Main String
hai gat hai cse
Enter the Pattern String
hai
Enter the Replacement String
best
Resultant String is: best gat best cse
```

```
Enter the Main String
bset gat best cse
Enter the Pattern String
ise
Enter the Replacement String
ece
Pattern Not Found
Resultant String is: bset gat best cse
```

PROGRAM 3

Design, Develop and Implement a menu driven Program in C for the following operations on STACK of Integers (Array Implementation of Stack with maximum size MAX)

a. ***Push* an Element on to Stack**
b. ***Pop* an Element from Stack**
c. **Demonstrate how Stack can be used to check *Palindrome***
d. **Demonstrate *Overflow* and *Underflow* situations on Stack**
e. **Display the status of Stack**
f. **Exit**

Support the program with appropriate functions for each of the above operations.

ABOUT THE EXPERIMENT:

A stack is a linear data structure. It follows **Last In First Out (LIFO)** principle commonly used in most programming languages. It is named stack as it behaves like a real-world stack, for example − deck of cards or pile of plate set.

This feature makes it LIFO data structure., the element which is placed (inserted or added) last is accessed first. In stack terminology, insertion operation is called **PUSH** operation and removal operation is called **POP**operation.
Below given diagram tries to depict a stack and its operations

A stack can be implemented by means of Array, Structure, Pointer and Linked-List. Stack can either be a fixed size one or it may have a sense of dynamic resizing. Here, we are going to implement stack using arrays which makes it a fixed size stackimplementation.

Basic Operations:

- **push()** - pushing (storing) an element on thestack.
- **pop()** - removing (accessing) an element from thestack.

To use a stack efficiently we need to check status of stack as well. For the same purpose, the following functionality is added to stacks;

- **isFull()** − check if stack is full.
- **isEmpty()** − check if stack is empty.

OBJECTIVES:

- Implementation stack data structure
- Application of stack(Checking stack is palindrome or not)

REAL TIME APPLICATIONS:

- *Reversing data (say string), Palindrome check*
- *Backtracking*
- *Recursion*
- *Converting decimal to binary*
- Infix to postfix conversion
- Infix to prefix conversion
- Evaluating postfix expression
- Evaluating prefix expression

Source Code:

```
#include<stdlib.h>
#include<stdio.h>
#define max_size 5
int stack[max_size],top=-1;
/*Function Prototype*/
void push();
void pop();
void display();
voidpali();
int main()
{
        int choice;
        while(1)
        {
                printf("\n\n--------STACK OPERATIONS ---------- \n");
                printf("1.Push\n");
                printf("2.Pop\n");
                printf("3.Palindrome\n");
                printf("4.Display\n");
                printf("5.Exit\n");
                printf(" ---------------------- ");
        printf("\nEnter your choice:\t");
                scanf("%d",&choice);
                switch(choice)
                {
                        case 1: push(); //to insert an item into stack
                                break;
                        case 2: pop(); //to delete an item from stack
                                break;
                        case 3: pali(); //to check palindrome or not
                                break;
                        case 4: display(); //to display items in stack
                                break;
                        case 5: exit(0);//exit the program
                                break;
                        default:printf("\n Invalid choice:\n");
```

```
                                        break;
                        }
                }
                return 0;
}
void push() //Inserting element into the stack
{
        intitem,n;
        if(top==(max_size-1))
        {
                printf("\nStack Overflow:");
        }
        else
        {
                printf("Enter the element to be inserted:\t");
                scanf("%d",&item);
                top=top+1;
                stack[top]=item;
        }
}

void pop() //deleting an element from the stack
{
        int item;
        if(top==-1)
        {
                  printf("Stack Underflow:");
        }
        else
        {
                item=stack[top];
                top=top-1;
                printf("\nThepoped element: %d\t",item);
        }
}
voidpali()      //checking whether palindrome or not
{
        intnum[10],rev[10],i=0,k,flag=1;
        k=top;
        while(k!=-1)
        {
                num[i++]=stack[k--];
        }
        for(i=0;i<=top;i++)
        {
                if(num[i]==stack[i])
                        continue;
                else
        }               flag=0;
```

```
        if(top!=-1)
        {
                if(flag)
                        printf("It is palindrome number\n");
                else
                        printf("It is not a palindrome number\n");
        }
        else


                printf("Stack is Empty:");
}
void display() //Displaying the elements of stack
{
        inti;
        if(top==-1)
        {
                printf("\nStack is Empty:");
        }
        else
        {
                printf("\nThe stack elements are:\n" );
                for(i=top;i>=0;i--)
                {
                        printf("%d\t",stack[i]);
                }
        }
}
```

Expected Output

```
--------STACK OPERATIONS-----------
1.Push
2.Pop
3.Palindrome
4.Display
5.Exit
------------------------
Enter your choice:      1
Enter the element to be inserted:       8

--------STACK OPERATIONS-----------
1.Push
2.Pop
3.Palindrome
4.Display
5.Exit
------------------------
Enter your choice:      1
Enter the element to be inserted:       7

--------STACK OPERATIONS-----------
1.Push
2.Pop
3.Palindrome
4.Display
5.Exit
------------------------
Enter your choice:      1
Enter the element to be inserted:       6

--------STACK OPERATIONS-----------
1.Push
2.Pop
3.Palindrome
4.Display
5.Exit
------------------------
Enter your choice:      1

Stack Overflow:
--------STACK OPERATIONS-----------
1.Push
2.Pop
3.Palindrome
4.Display
5.Exit
------------------------
Enter your choice:      4

The stack elements are:
6       7       8       7       6
--------STACK OPERATIONS-----------
1.Push
2.Pop
3.Palindrome
4.Display
5.Exit
------------------------
Enter your choice:      3
It is palindrome number


--------STACK OPERATIONS-----------
1.Push
2.Pop
3.Palindrome
4.Display
5.Exit
------------------------
Enter your choice:      2

The poped element: 6

--------STACK OPERATIONS-----------
1.Push
2.Pop
3.Palindrome
4.Display
5.Exit
------------------------
Enter your choice:      4

The stack elements are:
7       8       7       6
```

```
--------STACK OPERATIONS-----------
1.Push
2.Pop
3.Palindrome
4.Display
5.Exit
------------------------
Enter your choice:      3
It is not a palindrome number


--------STACK OPERATIONS-----------
1.Push
2.Pop
3.Palindrome
4.Display
5.Exit
------------------------
Enter your choice:      2

The poped element: 7
--------STACK OPERATIONS-----------
1.Push
2.Pop
3.Palindrome
4.Display
5.Exit
------------------------
Enter your choice:      2

The poped element: 8

--------STACK OPERATIONS-----------
1.Push
2.Pop
3.Palindrome
4.Display
5.Exit
------------------------
Enter your choice:      2

The poped element: 7

--------STACK OPERATIONS-----------
1.Push
2.Pop
3.Palindrome
4.Display
5.Exit
------------------------
Enter your choice:      2

The poped element: 6

--------STACK OPERATIONS-----------
1.Push
2.Pop
3.Palindrome
4.Display
5.Exit
------------------------
Enter your choice:      2
Stack Underflow:
--------STACK OPERATIONS-----------
1.Push
2.Pop
3.Palindrome
4.Display
5.Exit
------------------------
Enter your choice:      5
```

PROGRAM 4

Design, develop and implement a Program in C for converting an Infix Expression to Postfix Expression. Program should support for both parenthesized and free parenthesized expressions with the operators: +, -, *, /, %(Remainder), ^(Power) and alphanumeric operands.

ABOUT THE EXPERIMENT:

To convert infix expression to postfix expression is **application of stack**, we will use stack data structure. By scanning the infix expression from left to right, when we will get any operand, simply add them to the postfix form, and for the operator and parenthesis, add them in the stack maintaining the precedence of them.

Infix: Operators are written in-between their operands.

Ex: X + Y

Prefix: Operators are written before their operands.

Ex: +X Y

postfix: Operators are written after their operands.

Ex: XY+

Stack Table for a*(b+c)*d

Token	[0]	[1]	[2]	[3]	Top	Output
#	#				0	
A	#				0	a
*	#	*			1	a
(	#	*	(		2	a
B	#	*	(		2	ab
+	#	*	(	+	3	ab
C	#	*	(	+	3	abc
)	#	*			1	abc+
*	#	*			1	abc+*
D	#	*			1	abc+*d
Eos	#				0	abc+*d*

Infix Expression**: a*(b+c)*d,** Postfix Expression**: abc+*d***

OBJECTIVES:

To learn how to use stack data structure to convert infix expression to postfix expression.

REAL TIME APPLICATIONS:

Infix expressions are readable and solvable by humans. We can easily distinguish the order of operators, and also can use the parenthesis to solve that part first during solving mathematical expressions. The

computer cannot differentiate the operators and parenthesis easily, that's why postfix conversion is needed.

Source Code:

```
#include <ctype.h>
#include <stdio.h>
#define SIZE 50 /* Size of Stack */
char s[SIZE]; /* Global declarations */
int top = -1;
push(char elem) /* Function for PUSH operation */
{
        s[++top] = elem;
}
char pop() /* Function for POP operation */
{
        return (s[top--]);
}
intpr(char elem) /* Function for precedence */
{
        switch (elem)
        {
                case '#': return 0;
                case '(': return 1;
                case '+':
                case '-': return 2;
                case '*':
                case '/':
                case '%': return 3;
                case '^': return 4;
        }
}
void main() /* Main Program */
   {
        charinfx[50], pofx[50], ch, elem;
        inti = 0, k = 0;
        printf("\n\n enter the Infix Expression : ");
        gets(infx);
        push('#');
        while ((ch = infx[i++]) != '\0')
        {
                if (ch == '(')
                        push(ch);
                else if (isalnum(ch))
                        pofx[k++] = ch;
                else if (ch == ')')
                {
                        while (s[top] != '(')
```

```
                        pofx[k++] = pop();
                elem = pop();     /* Remove ( */
            }
            else     /* Operator */
            {
                while (pr(s[top]) >= pr(ch))
                    pofx[k++] = pop();
                push(ch);
            }
        }
        while (s[top] != '#') /* Pop from stack till empty */
            pofx[k++] = pop();
        pofx[k] = '\0'; /* Make pofx as valid string */
        printf("\n\n Given Infix Expn is: %s\n The Postfix Expn is:%s\n", infx, pofx);
}
```

EXPECTED OUTPUT:

```
enter the Infix Expression : a+(b-c)*d/e^f


Given Infix Expn is: a+(b-c)*d/e^f
The Postfix Expn is:abc-d*ef^/+


enter the Infix Expression : a*(b+c)/d%f$g


Given Infix Expn is: a*(b+c)/d%f$g
The Postfix Expn is:abc+*d/fg$%
```

PROGRAM 5

Design, develop and implement a Program in C for the following Stack Applications

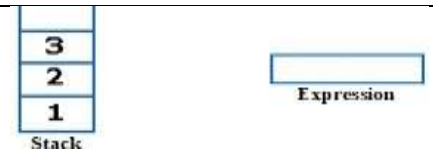
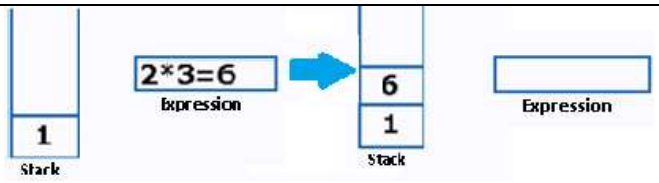
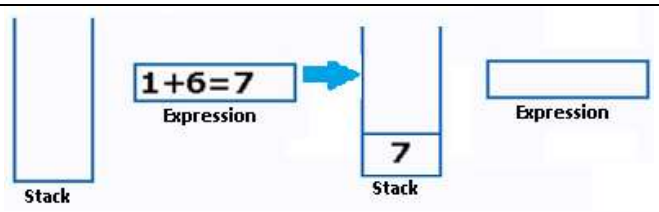
a. **Evaluation of Suffix expression with single digit operands and operators: +, -, *, /,%, ^**
b. **Solving Tower of Hanoi problem with n disks**

a. Evaluation of Suffix expression with single digit operands and operators: +, -, *, /,%, ^

ABOUT THE EXPERIMENT:

Evaluation of an expression is application of Stack.
Postfix/Suffix Expression: Operators are written after their operands. Ex: ab+, 52*
In normal algebra we use the infix notation like a/b$c. The corresponding postfix notation is abc$/

Example: Postfix String: 123*+

Initially the Stack is empty. Now, the first three characters scanned are 1,2 and 3, which are operands. Thus they will be pushed into the stack in that order.	→	
Next character scanned is "*", which is an operator. Thus, we pop the top two elements from the stack and perform the "*" operation with the two operands. The second operand will be the first element that is popped. The value of the expression(2*3) that has been evaluated(6) is pushed into the stack	→	
Next character scanned is "+", which is an operator. Thus, we pop the top two elements from the stack and perform the "+" operation with the two operands. The second operand will be the first element that is popped. The value of the expression(1+6) that has been evaluated(7) is pushed into the stack.	→	

Now, since all the characters are scanned, the remaining element in the stack (there will be only one element in thestack) will be returned.

Postfix String: 123*+ and Result: 7

OBJECTIVES:

Toevaluation of Suffix expression with single digit operands and operators

Source Code for program 5a:

```
#include<stdio.h>
#include<math.h>
#include<string.h>
int s[30],op1,op2;
int top=-1,i;
```

```
char p[30],sym;
int op(int op1,char sym,int op2)
{
        switch(sym)
        {
                case '+':return op1+op2;
                case '-':return op1-op2;
                case '*':return op1*op2;
                case '/':return op1/op2;
                case '%':return op1%op2;
                case '^':
                case '$':return pow(op1,op2);
        }
        return 0;
}
int main()
{
        printf("\nEnter the valid postfix exp:");
        scanf("%s",p);
        for(i=0;i<strlen(p);i++)
        {
                sym=p[i];
                if(sym>='0' && sym<='9')
                        s[++top]=sym-'0';
                else
                {
                        op2=s[top--];
                        op1=s[top--];
                        s[++top]=op(op1,sym,op2);
                }
        }

        printf("\nThe result is %d\n",s[top]);
   }
```

EXPECTED OUTPUT:

Enter the valid postfix exp: 651-4*23$/+ The result is 8	Enter the valid postfix exp: 861-*2/32^% The result is 2

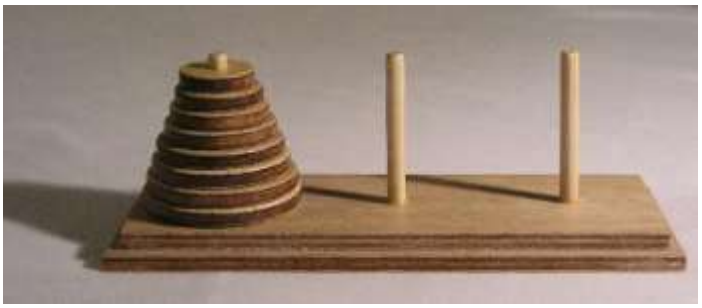
b. Solving Tower of Hanoi problem with ndisks.

ABOUT THE EXPERIMENT:

The **Tower of Hanoi** is a mathematical game or puzzle. It consists of three rods, and a number of disks of different sizes which can slide onto any rod. The puzzle starts with the disks in a neat stack in ascending order of size on one rod, the smallest at the top, thus making a conical shape. The objective of the puzzle is to move the entire stack to another rod, obeying the following simple rules:

- Only one disk can be moved at atime.
- Each move consists of taking the upper disk from one of the stacks and placing it on top of another stack i.e. a disk can only be moved if it is the uppermost disk on astack.
- No disk may be placed on top of a smallerdisk.

With three disks, the puzzle can be solved in seven moves. The minimum number of movesrequired to solve a Tower of Hanoi puzzle is 2^n - 1, where *n* is the number of disks.



OBJECTIVES:

To transfer a given disks from source to destination by using stack concepts with obeying the given rules:

REAL TIME APPLICATIONS:

- TOH is an application of stack.
- The **Tower of Hanoi** is commonly used by psychologists to research and examine problem solving skills. The recursive rule of the **Tower of Hanoi** is studied and applied in computer programming.
- Computer Games.

Source Code for program 5b:

```
#include<stdio.h>
int count=0,n;
int tower(int n,char s,char t,char d)
{
        if(n==1)
        {
                printf("\n Move disc 1 from %c to %c",s,d);
                count++;
                return 1;
        }
        tower(n-1,s,d,t);
        printf("\n Move disc %d from %c to %c",n,s,d);
        count++;
        tower(n-1,t,s,d);
}
int main( )
{
        printf("\n Enter the no. of discs:");
        scanf("%d",&n);
        tower(n,'A','B','C');
        printf("\n The no. of disc moves is:%d",count);
}
```

EXPECTED OUTPUT:

```
Enter the no. of discs: 1

Move disc 1 from A to C
The no. of disc moves is:1
Enter the no. of discs:2

Move disc 1 from A to B
Move disc 2 from A to C
Move disc 1 from B to C
The no. of disc moves is:3
```

```
Enter the no. of discs:3

Move disc 1 from A to C
Move disc 2 from A to B
Move disc 1 from C to B
Move disc 3 from A to C
Move disc 1 from B to A
Move disc 2 from B to C
Move disc 1 from A to C
The no. of disc moves is:7
```

PROGRAM 6

Design, develop and implement a menu driven Program in C for the following operations on Circular QUEUE of Characters (Array Implementation of Queue with maximum size MAX)

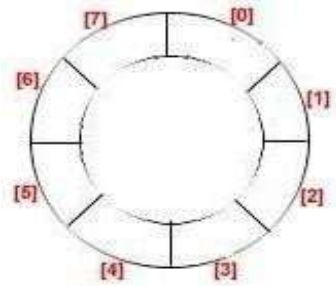
a. **Insert an Element on to Circular QUEUE**
b. **Delete an Element from Circular QUEUE**
c. **Demonstrate *Overflow* and *Underflow* situations on Circular QUEUE**
d. **Display the status of Circular QUEUE**
e. **Exit**

Support the program with appropriate functions for each of the above operations.

ABOUT THE EXPERIMENT:

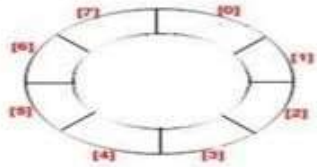
Circular queue is a linear data structure. It follows**First In First Out (FIFO) principle**. In circular queue the last node is connected back to the first node to make a circle. Elements are added at the rear end and the elements are deleted at front end of the queue. The queue is considered as a circular queue when the positions 0 and MAX-1 are adjacent. Any position before front is also after rear.

A circular queue looks like

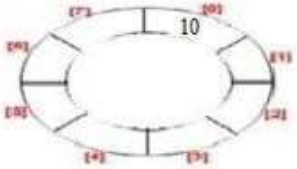


Consider the example with Circular Queue implementation:

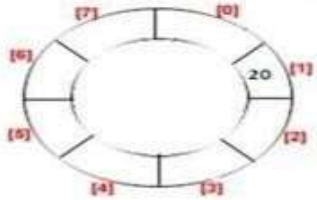
1) Initially: **Front = 0** and **rear = -1**



2) Add item 10 then **front = 0** and **rear =0.**

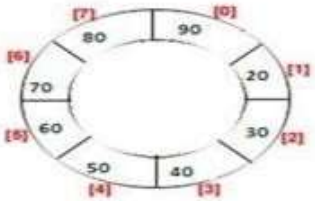


3) Now delete one item then **front = 1** and **rear =1.**



4) Like this now insert 30, 40, and 50,50,70,80 respectability then **front =1** and **rear = 7.**

5) Now in case of linear queue, we can not access 0 block for insertion but in circular queue next item will be inserted of 0 block then **front =0** and **rear = 0.**



OBJECTIVES:

To perform insert, delete operation on circular queue and Demonstrate Overflow and Underflow situations on Circular QUEUE

REAL TIME APPLICATIONS:

- Memory management: **circular queue** is used in memory management.
- Process Scheduling: A CPU **uses** a **queue** to schedule processes.
- Traffic Systems: Queues are also used in traffic systems

Insert the elements into circular queue. If queue is full give a message as "queue is overflow" otherwise insert new element based on formulae

rear = (rear + 1) % MAX

Delete an element from the circular queue. If queue is empty give a message as "queue is underflow" otherwise delete an element from Circular queue based on formulae

front = (front + 1) % MAX

Display the contents of the queue (You can display queue elements based on number of elements present in queue orbased on rear and front pointer)

Source Code:

```
#include<stdio.h>
#include<stdlib.h>
#define SIZE 5
char q[SIZE];
int i,r=-1;
int option,f=0;
int j,count=0;
void insert()
{
        if(count==SIZE)
                printf("\n Q is Full\n");
        else
        {
                 r=(r+1)%SIZE;
                printf("\nEnter the item:");
                scanf(" %c",&q[r]);
                count++;
        }}
void delet()
{
        if(count==0)
                printf("\nQ is empty\n");
         else
         {
                printf("\nDeleted item is: %c",q[f]);
                count--;
                 f=(f+1)%SIZE;
```

```
        }
}
void display()
{
        if(count==0)
                printf("\nQ is Empty\n");
        else
        {
                i=f;
                for(j=1;j<=count;j++)
                {

                        printf(" %c",q[i]);
                        i=(i+1)%SIZE;
                }
        }
}
int main()
{
        for(;;)
        {
                printf("\n1.Insert  2.Delete\n 3.Display 4.Exit");
                printf("\nEnter your option:");
                scanf("%d",&option);
                switch(option)
                {
                        case 1: insert();//Inserting items to Queue
                              break;
                        case 2: delet(); //Deleting items from Queue
                        break;
                        case 3:display();         //Displaying items from Queue
                         break;
                        default:exit(0);
                }
        }
}
```

EXPECTED OUTPUT:

```
1.Insert  2.Delete  3.Display 4.Exit
Enter your option:2

Q is empty

1.Insert  2.Delete  3.Display 4.Exit
Enter your option:3

Q is Empty
1.Insert  2.Delete  3.Display 4.Exit
Enter your option:1

Enter the item: A

1.Insert  2.Delete  3.Display 4.Exit
Enter your option:1

Enter the item: B

1.Insert  2.Delete  3.Display 4.Exit
Enter your option:1

Enter the item: C

1.Insert  2.Delete  3.Display 4.Exit
Enter your option:1

Enter the item: D

1.Insert  2.Delete  3.Display 4.Exit
Enter your option:1

Enter the item: E

1.Insert  2.Delete  3.Display 4.Exit
Enter your option:1

 Q is Full
```

```
1.Insert  2.Delete  3.Display 4.Exit
Enter your option:3
A B C D E
1.Insert  2.Delete  3.Display 4.Exit
Enter your option:2

Deleted item is: A
1.Insert  2.Delete  3.Display 4.Exit
Enter your option:2

Deleted item is: B
1.Insert  2.Delete  3.Display 4.Exit
Enter your option:2

Deleted item is: C
1.Insert  2.Delete  3.Display 4.Exit
Enter your option:3
 D E
1.Insert  2.Delete  3.Display 4.Exit
Enter your option:1

Enter the item: A

1.Insert  2.Delete  3.Display 4.Exit
Enter your option:1

Enter the item: B

1.Insert  2.Delete  3.Display 4.Exit
Enter your option:3
 D E A B
1.Insert  2.Delete  3.Display 4.Exit
Enter your option:1

Enter the item: C

1.Insert  2.Delete  3.Display 4.Exit
Enter your option:1

 Q is Full

1.Insert  2.Delete  3.Display 4.Exit
Enter your option:4
```

PROGRAM 7

Design, Develop and Implement a menu driven Program in C for the following operations on Singly Linked List (SLL) of Student Data with the fields: *USN, Name, Branch, Sem, PhNo*

a. **Create a SLL of N Students Data by using *front insertion*.**
b. **Display the status of SLL and count the number of nodes in it**
c. **Perform Insertion and Deletion at End of SLL**
d. **Perform Insertion and Deletion at Front of SLL (Demonstration of stack)**
e. **Exit**

ABOUT THE EXPERIMENT:

Linked list is a dynamic data structure whose length can be vary dramatically as elements are inserted and removed. Linked list are different from arrays:

- An array is a static data structure. This means the length of array cannot be altered at run time. While, a linked list is a dynamic data structure.
- In an array, all the elements are kept at consecutive memory locations while in a linked list the elements (or nodes) may be kept at any location but still connected to each other.

Linked List is a linear data structure and it is very common data structure which consists of group of nodes in a sequence which is divided in two parts. Each node consists of its own data and the address of the next node and forms a chain. Linked Lists are used to create trees andgraphs.

- They are a dynamic in nature which allocates the memory whenrequired.
- Insertion and deletion operations can be easilyimplemented.
- Stacks and queues can be easilyexecuted.
- Linked List reduces the accesstime.
- Linked lists are used to implement stacks, queues, graphs,etc.
- Linked lists let you insert elements at the beginning,between and end of thelist.
- In Linked Lists we don't need to know the size inadvance

Singly Linked List: Singly linked lists contain nodes which have a data part as well as an address part i.e. next, which points to the next node in sequence of nodes and the last node contains the NULL reference. The operations we can perform on singly linked lists are insertion, deletion and traversal. In this the elements can be placed anywhere in the heap memory unlike array which uses contiguous locations.

Basic operations of a singly-linked list are:

- **Insert –** Inserts a new element at the end of thelist.
- **Delete –** Deletes any node from thelist.
- **Find –** Finds any node in thelist and **Print –** Prints thelist.

OBJECTIVES:

To perform insert, delete and search operation on newly created linked list and Demonstration of stack

REAL TIME APPLICATIONS:

- we use undo button (Ctrl+x) in keyboard that's the best example we can't use the same button for redo we have Ctrl+y for that.
- Implementation of stacks and queues
- Implementation of graphs : Adjacency list representation of graphs is most popular which is uses linked list to store adjacent vertices.
- Dynamic memory allocation : We use linked list of free blocks.
- Maintaining directory of names
- Performing arithmetic operations on long integers
- Manipulation of polynomials by storing constants in the node of linked list
- Representing sparse matrices

The node of a linked list is a structure with fields data (which stored the value of the node) and*next (which is a pointer of type node that stores the address of the next node).Two nodes *start (which always points to the first node of the linked list) and *temp (which is used to point to the last node of the linked list) are initialized. Initially temp = start andtemp->next = NULL. Here, we take the first node as a dummy node. The first node does not contain data, but it used because to avoid handling special cases in insert and delete functions.

FUNCTIONS

1. Insert – This function takes the start node and data to be inserted as arguments. New node is inserted at the end so, iterate through the list till we encounter the last node. Then, allocate memory for the new node and put data in it. Lastly, store the address in thenextfield of the new node as NULL.
2. Delete - This function takes the start node (as pointer) and data to be deleted as arguments. Firstly, go to the node for which the node next to it has to bedeleted,
If that node points to NULL (i.e.pointer->next=NULL) then the element to be deleted is not present in the list.
Else, now pointer points to a node and the node next to it has to be removed, declare a temporary node (temp) which points to the node which has to be removed. Store the address of the node next tothe temporary node in the next field of the node pointer (pointer->next= temp->next). Thus, by breaking the link we removed the node which is next to the pointer (which is also temp). Because wedeleted the node, we no longer require the memory used for it, free() will deallocate the memory.
3. Find - This function takes the start node (as pointer) and data value of the node (key) to be found as arguments. First node is dummy node so, start with the second node. Iterate through the entire linked list and search for the key Until next field of the pointer is equal to NULL, check if pointer->data = key.

If it is then the key is found else, move to the next node and search (pointer = pointer -> next). If key is not found return 0 else return 1.

4. Print - function takes the start node (as pointer) as an argument. If pointer = NULL, then there is no element in the list. Else, print the data value of the node (pointer->data) and move to the next node by recursively calling the print function with pointer->next sent as an argument.

Source Code:

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
struct student
{
        char usn[11];
        char name[25];
        int sem;
        char branch[5];
        unsigned long long phno;
};
typedefstruct student STUD;
struct node
{  char usn[11];
        char name[25];
        int sem;
        char branch[5];
        unsigned long long phno;
        struct node *next;
};
typedefstruct node NODE;
NODE *first;
NODE* copyNode(STUD s)
{       NODE * temp;
        temp= (NODE *)malloc(sizeof(NODE));
         if(temp==NULL)
        {
                printf("Memory cannot be allocated\n");
        }
        else
        {       strcpy(temp->usn,s.usn);
                strcpy(temp->name, s.name);
                strcpy(temp->branch, s.branch);
                temp->sem=s.sem;
                temp->phno=s.phno;
                temp->next=NULL;
                return temp;
}       }
void addrear(STUD s)
{
```

```
        NODE *temp,*cur;
        temp=copyNode(s) ;
        if(first==NULL)
        {
                first=temp; return;
        }
        cur=first;
        while(cur->next != NULL)
        {
                 cur=cur->next;
        }
        cur->next =temp;
        return ;
}
void addfront(STUD s)
{
         NODE *temp;
        temp=copyNode(s); //ssn,name, dept, design,salary, pno);
        if (first== NULL)
        {
                first=temp;
        }
        else
        {
                temp->next=first;
                first=temp;
        }

         return ;
}
void display(NODE *temp)
{
        printf("%s \t", temp->usn);
        printf("%s \t", temp->name);
        printf("%s \t", temp->branch);
        printf("%d \t",temp->sem);
        printf("%llu \n", temp->phno);
}
void deletefront()
{
        NODE *temp; int num;
        temp=first;
        if(first==NULL)

        {
                printf("List is Empty"); return;
        }
        if(first->next==NULL)
                first=NULL; else
```

```
        {
                first=first->next;
         }
        printf("Deleted Node is:\n");
        display(temp);
        free(temp);
        return ;
}
void deleterear()
{
        NODE *cur, *prev;
        cur=first;
        prev=NULL;
        if(first==NULL)
        {
                printf("List is Empty"); return;
        }
        if(first->next==NULL)
        {
                display(cur);
                first=NULL;
                free(cur); return;
        }

        while(cur->next!=NULL)
        {
                prev=cur;
                cur=cur->next;
        }
        prev->next=NULL;
        printf("Deleted Node is:\n");
        display(cur);
        free(cur);
        return;
}
void  displayList()
{
        NODE *r;
        r=first;
        printf("USN\tName\tBrh\tSem\tPhone\n");
        if(r==NULL)
                return;
        while(r!=NULL)
        {
                display(r);
                r=r->next;
        }
        printf("\n");
}
```

```
STUD input()
{
        STUD s;
        printf("Enter USN : ");
        scanf("%s",s.usn);
        printf("Enter Name : ");
        scanf("%s",s.name);
        printf("Enter Branch: ");
        scanf("%s",s.branch);
        printf("Enter Sem:");
        scanf("%d",&s.sem);
        printf("Enter Phone no : ");
        scanf("%llu",&s.phno);
        return s;
}
int count()
{
   NODE *n;
   int c=0;
   n=first;
   while(n!=NULL)
   {    n=n->next;
        c++;
   }
   return c;
}
int  main()
{
   STUD s; int i, ch, n;
   first=NULL;
   while(1)
   {
        printf("\nList Operations\n");
        printf("===============\n");
        printf("1.Create List of n students by using front Insert\n");
        printf("2.Display the status and count the nodes\n");
        printf("3.Perform Insertion and Deletion at End of SLL\n");
        printf("4.Perform Insertion and Deletion at Front of SLL\n");
        printf("5.Exit\n");
        printf("Enter your choice : ");
        scanf("%d",&i);
        switch(i)
        {
                case 1 : printf("Enter the number of students\n");
                         scanf("%d",&n);
                         for(i=1;i<=n;i++)
                         {
                                 printf("\nEnter the details of student %d\n",i);
```

```
                    s=input();
                    addfront(s);
                }
                break;


        case 2 : if(first==NULL)
                {
                     printf("List is Empty\n");
                }
                else
                {
                    printf(" Node Count=%d\t & Elements in the list are : \n", count());
                    displayList();
                }
                break;


        case 3 : printf(" 1. Insert at End and 2 Delete From End=");
                scanf("%d",&ch);
                if(ch==1)
                {
                    s=input();
                    addrear(s);
                }
                else if(ch==2)
                     deleterear();
                 else
                     printf(" Sorry wrong operation\n");
                break;


        case 4 : printf(" 1. Insert at Front and 2  Delete From Front=");
                 scanf("%d",&ch);
                if(ch==1)
                {
                    s=input();
                    addfront(s);
                }
                else if(ch==2)
                    deletefront();
                else

                 break; printf(" Sorry wrong operation\n");

        case 5:  exit (0);
        default : printf("Invalid option\n");
    }}
return 0;}
```

EXPECTED OUTPUT:

```
List Operations
==============
1.Create List of n students by using front Insert
2.Display the status and count the nodes
3.Perform Insertion and Deletion at End of SLL
4.Perform Insertion and Deletion at Front of SLL
5.Exit
Enter your choice : 1
Enter the number of students
2

Enter the details of student 1
Enter USN   : 001
Enter Name  : shakuntaladevi
Enter Branch: cse
Enter Sem:3
Enter Phone no : 1111111111

Enter the details of student 2
Enter USN   : 002
Enter Name  : indranooyi
Enter Branch: cse
Enter Sem:3
Enter Phone no : 2222222222

List Operations
==============
1.Create List of n students by using front Insert
2.Display the status and count the nodes
3.Perform Insertion and Deletion at End of SLL
4.Perform Insertion and Deletion at Front of SLL
5.Exit
Enter your choice : 2
 Node Count=2    & Elements  in the list are :
USN     Name    Brh     Sem     Phone
002     indranooyi      cse     3       2222222222
001     shakuntaladevi  cse     3       1111111111
```

```
List Operations
==============
1.Create List of n students by using front Insert
2.Display the status and count the nodes
3.Perform Insertion and Deletion at End of SLL
4.Perform Insertion and Deletion at Front of SLL
5.Exit
Enter your choice : 3
 1. Insert at End and 2 Delete From End=1
Enter USN   : 003
Enter Name  : chandakochhar
Enter Branch: cse
Enter Sem:3
Enter Phone no : 3333333333

List Operations
==============
1.Create List of n students by using front Insert
2.Display the status and count the nodes
3.Perform Insertion and Deletion at End of SLL
4.Perform Insertion and Deletion at Front of SLL
5.Exit
Enter your choice : 2
 Node Count=3    & Elements  in the list are :
USN     Name    Brh     Sem     Phone
002     indranooyi      cse     3       2222222222
001     shakuntaladevi  cse     3       1111111111
003     chandakochhar   cse     3       3333333333
```

```
List Operations
===============
1.Create List of n students by using front Insert
2.Display the status and count the nodes
3.Perform Insertion and Deletion at End of SLL
4.Perform Insertion and Deletion at Front of SLL
5.Exit
Enter your choice : 4
 1. Insert at Front and 2  Delete From Front=1
Enter USN   : 004
Enter Name  : shikhasharma
Enter Branch: cse
Enter Sem:3
Enter Phone no : 4444444444

List Operations
===============
1.Create List of n students by using front Insert
2.Display the status and count the nodes
3.Perform Insertion and Deletion at End of SLL
4.Perform Insertion and Deletion at Front of SLL
5.Exit
Enter your choice : 2
 Node Count=4    & Elements  in the list are :
USN     Name    Brh     Sem     Phone
004     shikhasharma    cse     3       4444444444
002     indranooyi      cse     3       2222222222
001     shakuntaladevi  cse     3       1111111111
003     chandakochhar   cse     3       3333333333
```

```
List Operations
===============
1.Create List of n students by using front Insert
2.Display the status and count the nodes
3.Perform Insertion and Deletion at End of SLL
4.Perform Insertion and Deletion at Front of SLL
5.Exit
Enter your choice : 4
 1. Insert at Front and 2  Delete From Front=2
Deleted Node is:
004     shikhasharma    cse     3       4444444444
```

```
List Operations
===============
1.Create List of n students by using front Insert
2.Display the status and count the nodes
3.Perform Insertion and Deletion at End of SLL
4.Perform Insertion and Deletion at Front of SLL
5.Exit
Enter your choice : 2
 Node Count=3    & Elements  in the list are :
USN     Name    Brh     Sem     Phone
002     indranooyi      cse     3       2222222222
001     shakuntaladevi  cse     3       1111111111
003     chandakochhar   cse     3       3333333333


List Operations
===============
1.Create List of n students by using front Insert
2.Display the status and count the nodes
3.Perform Insertion and Deletion at End of SLL
4.Perform Insertion and Deletion at Front of SLL
5.Exit
Enter your choice : 3
 1. Insert at End and 2 Delete From End=2
Deleted Node is:
003     chandakochhar   cse     3       3333333333

List Operations
===============
1.Create List of n students by using front Insert
2.Display the status and count the nodes
3.Perform Insertion and Deletion at End of SLL
4.Perform Insertion and Deletion at Front of SLL
5.Exit
Enter your choice : 2
 Node Count=2    & Elements  in the list are :
USN     Name    Brh     Sem     Phone
002     indranooyi      cse     3       2222222222
001     shakuntaladevi  cse     3       1111111111
```

```
List Operations
===============
1.Create List of n students by using front Inse
2.Display the status and count the nodes
3.Perform Insertion and Deletion at End of SLL
4.Perform Insertion and Deletion at Front of SL
5.Exit
Enter your choice : 5
```

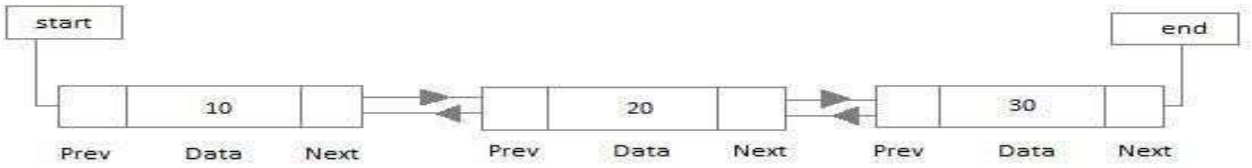
PROGRAM 8

Design, develop and implement a menu driven Program in C for the following operations on Doubly Linked List (DLL) of Employee Data with the fields: *SSN,Name, Dept, Designation, Sal, PhNo*

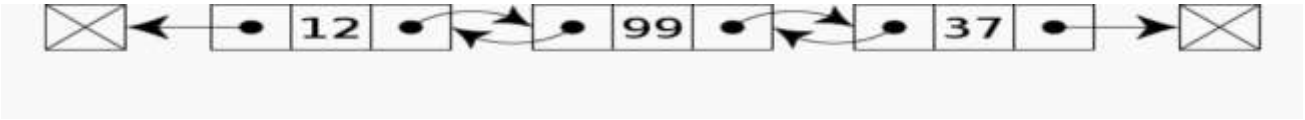
a. **Create a DLL of N Employees Data by using *end insertion*.**
b. **Display the status of DLL and count the number of nodes in it**
c. **Perform Insertion and Deletion at End of DLL**
d. **Perform Insertion and Deletion at Front of DLL**
e. **Demonstrate how this DLL can be used as Double Ended Queue**
f. **Exit**

ABOUT THE EXPERIMENT:

Doubly Linked List: In a doubly linked list, each node contains two links the first link points to the previous node and the next link points to the next node in the sequence.



In computer science, a **doubly linked list** is a linked data structure that consists of a set of sequentially linked records called nodes. Eachnode contains two fields, called *links*, thatare references to the previous and to the next node in the sequence of nodes. The beginning and ending nodes' **previous** and**next** links, respectively, point to some kind of terminator, typically a sentinel node or null, to facilitate traversal of the list. If there is only one sentinel node, then the list is circularly linked via the sentinel node. It can be conceptualized as two singly linked lists formed from the same data items, but in opposite sequentialorders.



A doubly linked list whose nodes contain three fields: an integer value, the link to the next node, and the link to the previous node.The two node links allow traversal of the list in either direction. While adding or removing a node in a doubly linked list requires changing more links than the same operations on a singly linked list, the operations are simpler and potentially more efficient (for nodes other than first nodes) because there is no need to keep track of the previous node during traversal or noneed to traverse the list to find the previous node, so that its link can bemodified.

OBJECTIVES:

To perform insert, delete and search operation on newly created doubly linked list for n employee and Demonstrate how this DLL can be used as Double Ended Queue

REAL TIME APPLICATIONS:

- Doubly linked list can be used in navigation systems where both front and back navigation is required.

- It is used by browsers to implement backward and forward navigation of visited web pages i.e. back and forward button.
- It is also used by various application to implement Undo and Redo functionality.
- In our media player we have to play next and previous songs we can go any where.
- a lift in a buildings
- Applications that have a Most Recently Used list (a linked list of file names)

The node of a linked list is a structure with fields data (which stored the value of the node), *previous (which is a pointer of type node that stores the address of the previous node) and *next (which is a pointer of type node that stores the address of the next node).
Two nodes *start (which always points to the first node of the linked list) and *temp (which is used to point to the last node of the linked list) are initialized. Initially temp = start, temp->prevoius = NULL and temp->next = NULL. Here, we take the first node as a dummy node. The first node does not contain data, but it used because to avoid handling special cases in insert and delete functions.

Functions:

1. **Insert –** This function takes the start node and data to be inserted as arguments. New node is inserted at the end so, iterate through the list till we encounter the last node. Then, allocate memory for the new node and put data in it. Lastly, store the address of the previous node in the previous field of the new node and address in the next field of the new node as NULL.
2. **Delete -** This function takes the start node (as pointer) and data to be deleted as arguments. Firstly, go to the node for which the node next to it has to be deleted,
If that node points to NULL (i.e. pointer->next=NULL) then the element to be deleted is not present in the list.
Else, now pointer points to a node and the node next to it has to be removed, declare a temporary node (temp) which points to the node which has to be removed. Store the address of the node next to the temporary node in the next field of the node pointer (pointer->next = temp->next) and also link the pointer and the node next to the node to be deleted (temp->prev = pointer). Thus, by breaking the link we removed the node which is next to the pointer (which is also temp). Because we deleted the node, we no longer require the memory used for it, free() will de-allocate the memory.
3. **Find** - This function takes the start node (as pointer) and data value of the node (key) to be found as arguments. First node is dummy node so, start with the second node. Iterate through the entire linked list and search for the key. Until next field of the pointer is equal to NULL, check if pointer->data = key. If it is then the key is found else, move to the next node and search (pointer = pointer -> next). If key is not found return 0, else return 1.
4. **Print -** function takes the start node (as pointer) as an argument. If pointer = NULL, then there is no element in the list. Else, print the data value of the node (pointer->data) and move to the next node by recursively calling the print function with pointer->next sent as an argument.

Source Code:

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
struct employee
{
        char ssn[11];
        char name[21];
        char dept[15];
        char design[15];
        int  salary;
        unsigned long long  phno;
};
typedef struct employee EMP;

struct node
{
         char ssn[11];
        char name[21];
        char dept[15];
        char design[15];
        int  salary;
        unsigned long long phno;
        struct node *next;
        struct node *prev;
};
typedef struct node NODE;
NODE *first;
NODE* copyNode(EMP e)
{
        NODE * temp;
        temp= (NODE *)malloc(sizeof(NODE));
         if(temp==NULL)
        {
                printf("Memory cannot be allocated\n");
        }
        else
        {
                 strcpy(temp->ssn,e.ssn);
                strcpy(temp->name, e.name);
                strcpy(temp->dept, e.dept);
                strcpy(temp->design, e.design);
                temp->salary=e.salary;
                temp->phno=e.phno;
                temp->next=NULL;
                temp->prev=NULL;
                return temp;
        }
```

```
}

void addrear(EMP e)
{
        NODE *temp,*cur, *prev;
        temp=copyNode(e) ;
        if(first==NULL)
        {
                first=temp; return;
        }
        cur=first; prev=NULL;
        while(cur->next != NULL)
        {
                prev=cur;
                cur=cur->next;
        }
        cur->next =temp;
        temp->prev=cur;
        return ;
}

void addfront(EMP e)
{
        NODE *temp;
        temp=copyNode(e);
        if (first== NULL)
        {
                first=temp;
        }
        else
        {
                 temp->next=first;
                first->prev=temp;
                first=temp;
        }

         return ;
}
void display(NODE *r)
{
        printf("%s\t", r->ssn);
        printf("%s\t", r->name);
        printf("%s\t", r->dept);
        printf("%s\t",r->design);
        printf("%d\t",r->salary);
        printf("%llu\n", r->phno);
}

void deletefront(){
```

```
        NODE *temp; int num;
        temp=first;
        if(first==NULL)
        {
                printf(" List is Empty \n");return ;
        }
        if(first->next==NULL)
                first=NULL;
        else
        {       first=first->next;
                first->prev=NULL;
        }
        printf("SSN\tName\tDept\tDesignation\tSalary\tPhone\n");
        display(temp);
        free(temp);
        return ;
}

void deleterear()
{
        NODE *cur, *prev;
        cur=first;
        prev=NULL;
        if(first==NULL)
        {
                printf(" List is Empty \n");return ;
        }
        if(first->next==NULL)
        {
                first=NULL;
        }
        else
        {
                while(cur->next!=NULL)
                {
                        prev=cur;
                        cur=cur->next;
                }
                prev->next=NULL;
        }
        printf("SSN\tName\tDept\tDesignation\tSalary\tPhone\n");
        display(cur);
        free(cur);
        return;
}

void  displayList()
{
        NODE *r;
```

```
        r=first;
        if(r==NULL)
        {
                return;
         }
        printf("SSN\t Name\t Dept\t Designation\t salary\tPhone\n");
        while(r!=NULL)
        {
                display(r);
                r=r->next;
         }
        printf("\n");
}

int count()
{
   NODE *n;
   int c=0;
   n=first;
   while(n!=NULL)
   {
        n=n->next;
        c++;
   }
   return c;
}

EMP input()
{
        EMP e;
        printf("Enter SSN : ");
        scanf("%s",e.ssn);
        printf("Enter Name : ");
        scanf("%s",e.name);
        printf("Enter dept: ");
        scanf("%s",e.dept);
        printf("Enter Designation :");
        scanf("%s",e.design);
        printf("Enter Salary:");
        scanf("%d",&e.salary);
        printf("Enter Phone no : ");
        scanf("%llu",&e.phno);
        return e;
}

int  main()
{
        EMP e;
        int i, ch, n;
```

```
first=NULL;
while(1)
{
        printf("\nList Operations\n");
        printf("===============\n");
        printf("1.Create a DLL of N Employees Data by using end insertion\n");
        printf("2.Display the status of DLL and count the number of nodes in it\n");
        printf("3.Perform Insertion and Deletion at End of DLL\n");
        printf("4.Perform Insertion and Deletion at Front of DLL\n");
        printf("5.Demonstration of this DLL as Double Ended Queue\n");
        printf("6.Exit\n");
        printf("Enter your choice : ");
        scanf("%d",&i);
        switch(i)
         {
                case 1 : printf("Enter the number of Employees\n");
                        scanf("%d",&n);
                        for(i=1;i<=n;i++)
                        {




                        }
                   break; printf("\nEnter the details of Employee %d\n",i);e=input();
                   addrear(e);
                 case 2 : if(first==NULL)
                        {
                                printf("List is Empty\n");}
                        else
                        {
                          printf(" Node Count=%d\t & Elements  in the list are : \n", count());
                          displayList();
                        }
                        break;
                case 3: printf(" 1. Insert at End and 2  Delete From End=");
                        scanf("%d",&ch);
                        if(ch==1)
                        {
                                e=input();
                                addrear(e);
                        else if(ch==2)
                                deleterear();
                        else
                                 printf(" Sorry wrong operation\n");
                        break;


                case 4:  printf(" 1. Insert at Front and 2  Delete From Front=");
                        scanf("%d",&ch);
```

```
                        if(ch==1)
                        {
                                e=input();
                                addfront(e);
                        }
                        else if(ch==2)
                                deletefront();
                        else
                                printf(" Sorry wrong operation\n");
                        break;


                case 5: printf("This DLL can be used as Double Ended Queue by inserting
                        and deleting from both ends \n");
                        break;

                case 6  :return 0;
                default :printf("Invalid option\n");
                }
        }
        return 0;
}
```

EXPECTED OUTPUT

```
List Operations
==============
1.Create a DLL of N Employees Data by using end insertion
2.Display the status of DLL and count the number of nodes in it
3.Perform Insertion and Deletion at End of DLL
4.Perform Insertion and Deletion at Front of DLL
5.Demonstration of this DLL as Double Ended Queue
6.Exit
Enter your choice : 1
Enter the number of Employees
3

Enter the details of Employee 1
Enter SSN   : 111
Enter Name  : ram
Enter dept: sales
Enter Designation :manager
Enter Salary:50000
Enter Phone no : 9999999999

Enter the details of Employee 2
Enter SSN   : 222
Enter Name  : sham
Enter dept: design
Enter Designation :manager
Enter Salary:55000
Enter Phone no : 8888888888

Enter the details of Employee 3
Enter SSN   : 3
Enter Name  : sunny
Enter dept: market
Enter Designation :manager
Enter Salary:60000
Enter Phone no : 7777777777

List Operations
==============
1.Create a DLL of N Employees Data by using end insertion
2.Display the status of DLL and count the number of nodes in it
3.Perform Insertion and Deletion at End of DLL
4.Perform Insertion and Deletion at Front of DLL
5.Demonstration of this DLL as Double Ended Queue
6.Exit
Enter your choice : 2
 Node Count=3    & Elements  in the list are :
SSN      Name    Dept    Designation     salary Phone
111      ram     sales   manager 50000   9999999999
222      sham    design  manager 55000   8888888888
3        sunny   market  manager 60000   7777777777


List Operations
==============
1.Create a DLL of N Employees Data by using end insertion
2.Display the status of DLL and count the number of nodes in it
3.Perform Insertion and Deletion at End of DLL
4.Perform Insertion and Deletion at Front of DLL
5.Demonstration of this DLL as Double Ended Queue
6.Exit
Enter your choice : 3
 1. Insert at End and 2  Delete From End=1
Enter SSN   : 444
Enter Name  : jack
Enter dept: sales
Enter Designation :admin
Enter Salary:45000
Enter Phone no : 6666666666

List Operations
==============
1.Create a DLL of N Employees Data by using end insertion
2.Display the status of DLL and count the number of nodes in it
3.Perform Insertion and Deletion at End of DLL
4.Perform Insertion and Deletion at Front of DLL
5.Demonstration of this DLL as Double Ended Queue
6.Exit
Enter your choice : 4
 1. Insert at Front and 2  Delete From Front=1
Enter SSN   : 555
Enter Name  : rose
Enter dept: admin
Enter Designation :manager
Enter Salary:50000
Enter Phone no : 5555555555
```

```
List Operations
==============
1.Create a DLL of N Employees Data by using end insertion
2.Display the status of DLL and count the number of nodes in it
3.Perform Insertion and Deletion at End of DLL
4.Perform Insertion and Deletion at Front of DLL
5.Demonstration of this DLL as Double Ended Queue
6.Exit
Enter your choice : 2
 Node Count=5    & Elements  in the list are :
SSN      Name    Dept    Designation     salary Phone
555     rose    admin   manager 50000   5555555555
111     ram     sales   manager 50000   9999999999
222     sham    design  manager 55000   8888888888
3       sunny   market  manager 60000   7777777777
444     jack    sales   admin   45000   6666666666


List Operations
==============
1.Create a DLL of N Employees Data by using end insertion
2.Display the status of DLL and count the number of nodes in it
3.Perform Insertion and Deletion at End of DLL
4.Perform Insertion and Deletion at Front of DLL
5.Demonstration of this DLL as Double Ended Queue
6.Exit
Enter your choice : 3
 1. Insert at End and 2  Delete From End=2
SSN     Name    Dept    Designation     Salary  Phone
444     jack    sales   admin   45000   6666666666

List Operations
==============
1.Create a DLL of N Employees Data by using end insertion
2.Display the status of DLL and count the number of nodes in it
3.Perform Insertion and Deletion at End of DLL
4.Perform Insertion and Deletion at Front of DLL
5.Demonstration of this DLL as Double Ended Queue
6.Exit
Enter your choice : 4
 1. Insert at Front and 2  Delete From Front=2
SSN     Name    Dept    Designation     Salary  Phone
555     rose    admin   manager 50000   5555555555

List Operations
==============
1.Create a DLL of N Employees Data by using end insertion
2.Display the status of DLL and count the number of nodes in it
3.Perform Insertion and Deletion at End of DLL
4.Perform Insertion and Deletion at Front of DLL
5.Demonstration of this DLL as Double Ended Queue
6.Exit
Enter your choice : 2
 Node Count=3    & Elements  in the list are :
SSN      Name    Dept    Designation     salary Phone
111     ram     sales   manager 50000   9999999999
222     sham    design  manager 55000   8888888888
3       sunny   market  manager 60000   7777777777


List Operations
==============
1.Create a DLL of N Employees Data by using end insertion
2.Display the status of DLL and count the number of nodes in it
3.Perform Insertion and Deletion at End of DLL
4.Perform Insertion and Deletion at Front of DLL
5.Demonstration of this DLL as Double Ended Queue
6.Exit
Enter your choice : 3
 1. Insert at End and 2  Delete From End=2
SSN     Name    Dept    Designation     Salary  Phone
3       sunny   market  manager 60000   7777777777

List Operations
==============
1.Create a DLL of N Employees Data by using end insertion
2.Display the status of DLL and count the number of nodes in it
3.Perform Insertion and Deletion at End of DLL
4.Perform Insertion and Deletion at Front of DLL
5.Demonstration of this DLL as Double Ended Queue
6.Exit
Enter your choice : 4
 1. Insert at Front and 2  Delete From Front=2
SSN     Name    Dept    Designation     Salary  Phone
```

```
SSN      Name     Dept     Designation      Salary  Phone
222      sham     design   manager 55000    8888888888

List Operations
===============
1.Create a DLL of N Employees Data by using end insertion
2.Display the status of DLL and count the number of nodes in it
3.Perform Insertion and Deletion at End of DLL
4.Perform Insertion and Deletion at Front of DLL
5.Demonstration of this DLL as Double Ended Queue
6.Exit
Enter your choice : 4
 1. Insert at Front and 2  Delete From Front=2
 List is Empty

List Operations
===============
1.Create a DLL of N Employees Data by using end insertion
2.Display the status of DLL and count the number of nodes in it
3.Perform Insertion and Deletion at End of DLL
4.Perform Insertion and Deletion at Front of DLL
5.Demonstration of this DLL as Double Ended Queue
6.Exit
Enter your choice : 3
 1. Insert at End and 2  Delete From End=2
 List is Empty

List Operations
===============
1.Create a DLL of N Employees Data by using end insertion
2.Display the status of DLL and count the number of nodes in it
3.Perform Insertion and Deletion at End of DLL
4.Perform Insertion and Deletion at Front of DLL
5.Demonstration of this DLL as Double Ended Queue
6.Exit
Enter your choice : 2
List is Empty

List Operations
===============
1.Create a DLL of N Employees Data by using end insertion
2.Display the status of DLL and count the number of nodes in it
3.Perform Insertion and Deletion at End of DLL
4.Perform Insertion and Deletion at Front of DLL
5.Demonstration of this DLL as Double Ended Queue
6.Exit
Enter your choice : 5
This DLL can be used as Double Ended Queue by inserting and deleting from both ends

List Operations
===============
1.Create a DLL of N Employees Data by using end insertion
2.Display the status of DLL and count the number of nodes in it
3.Perform Insertion and Deletion at End of DLL
4.Perform Insertion and Deletion at Front of DLL
5.Demonstration of this DLL as Double Ended Queue
6.Exit
Enter your choice : 6
```

PROGRAM 9

Design, Develop and Implement a Program in C for the following operations on Singly Circular Linked List (SCLL) with header nodes

a. **Represent and Evaluate a Polynomial $P(x,y,z) = 6x^2y^2z-4yz^5+3x^3yz+2xy^5z-2xyz^3$**
b. **Find the sum of two polynomials POLY1(x,y,z) and POLY2(x,y,z) and store the result in POLYSUM(x,y,z)**

Support the program with appropriate functions for each of the above operations.

ABOUT THE EXPERIMENT:

Circular Linked List:

In the circular linked list the last node of the list contains the address of the first node and forms a circular chain.

Polynomial:

A polynomial equation is an equation that can be written in the form. $ax^n + bx^{n-1} + \ldots + rx + c = 0$, where a, b, . . . , r and c are constants. We call the largest exponent of x appearing in a non-zero termof a polynomial the "**degree"** of that polynomial.When a term contains both a number and a variable part, the number part is called the "coefficient". The coefficient on the leading term is called the **"leading"** coefficient.

a. **Represent and Evaluate a Polynomial $P(x,y,z) = 6x^2y^2z-4yz^5+3x^3yz+2xy^5z-2xyz^3$**

OBJECTIVES:

To compute a given polynomial of three variables using singly circular linked list.

REAL TIME APPLICATIONS:

- A ludo game has to come back give turn to first player after 4th player so it has to be circular.
- In our keyboard we have alt+tab key this is circular linked list it goes back on to first tab after last tab and we can go back also.

Source Code-Program 9a:

```
#include<stdio.h>
#include<stdlib.h>
#include<math.h>

int coef,px,py,pz, x,y,z,i;
int val;

struct node
{
        int coef,px, py,pz;
        struct node *next;
};
typedef struct node NODE;
NODE *first;

void insert(int coef,int px, int py, int pz)
{
        NODE *temp,*cur;
        temp= (NODE *)malloc(sizeof(NODE));
        temp->coef=coef; temp->px=px; temp->py=py; temp->pz=pz;
        if(first==NULL)
        {
                temp->next=temp;
                first=temp;  return;
        }
        if(first->next==first)
        {
                first->next=temp; temp->next=first;
        }
        cur=first;
        while(cur->next!=first)
        {
                cur=cur->next;
        }
        cur->next=temp;
        temp->next=first;
        return;
}


void display()
{
        NODE *cur;
        if(first==NULL)
        {
                printf("List is empty\n");return;
        }
```

```
        cur=first;
        while(cur->next!=first)
        {
                printf("%d ",cur->coef);
                printf(" x^%d",cur->px);
                printf(" y^%d",cur->py);
                printf(" z^%d + ",cur->pz);
                cur=cur->next;
         }
         printf("%d ",cur->coef);
         printf(" x^%d",cur->px);
         printf(" y^%d",cur->py);
         printf(" z^%d\n",cur->pz);
         return;
}

int  evaluate(int x, int y, int z)
{
        NODE *cur; int v,s=0, v1,v2,v3;
        if(first==NULL)
        {
                printf("List is empty\n");return 0;
        }
        cur=first;
        while(cur->next!=first)
        {
                v=cur->coef*pow(x, cur->px)*pow(y, cur->py)*pow(z,cur->pz);
                 s=s+v;
                cur=cur->next;
        }
        v=cur->coef*pow(x, cur->px)*pow(y, cur->py)*pow(z,cur->pz);
        s=s+v;
         return s;
}

int main()
{
        int coef,px,py,pz, x,y,z,i;
        int val;
        first=NULL;
        while(1)
        {
                printf("1. Insert polynomial at end\n");
                printf("2. Display\n");
                printf("3. Evaluate\n");
                printf("4. Exit\n");
                printf("Enter Choice= \t");
                scanf("%d",&i);
                switch(i)
```

```
                {
                        case 1 : printf("Enter Coefficient= \t");
                                 scanf("%d",&coef);
                                 printf("Enter powers of x y z values= \t");
                                 scanf("%d%d%d",&px, &py,&pz);
                                 insert(coef,px,py,pz);
                                 break;

                        case 2 : display();
                                 break;

                        case 3 : printf("\n Enter x y & z values for evaluation: \t");
                                 scanf("%d%d%d",&x,&y,&z);
                                 val=evaluate(x,y,z);
                                 printf("\nValue=%d\n",val);
                                 break;

                        case 4 :  return 0;

                        default : printf(" Wrong choice. Enter 1,2 3\n"); break;
                }
        }
}
```

EXPECTED OUTPUT:

```
1. Insert polynomial at end
2. Display
3. Evaluate
4. Exit
Enter Choice=   1
Enter Coefficient=      4
Enter powers of x y z values=   3 3 2
1. Insert polynomial at end
2. Display
3. Evaluate
4. Exit
Enter Choice=   1
Enter Coefficient=      5
Enter powers of x y z values=   3 2 2
1. Insert polynomial at end
2. Display
3. Evaluate
4. Exit
Enter Choice=   1
Enter Coefficient=      3
Enter powers of x y z values=   2 2 2
1. Insert polynomial at end
2. Display
3. Evaluate
4. Exit
Enter Choice=   1
Enter Coefficient=      6
Enter powers of x y z values=   1 1 1
1. Insert polynomial at end
2. Display
3. Evaluate
4. Exit
Enter Choice=   2
4  x^3 y^3 z^2 + 5  x^3 y^2 z^2 + 3  x^2 y^2 z^2 + 6  x^1 y^1 z^1
1. Insert polynomial at end
2. Display
3. Evaluate
4. Exit
Enter Choice=   3

 Enter x y & z values for evaluation:   3 2 1

Value=1548
1. Insert polynomial at end
2. Display
3. Evaluate
4. Exit
Enter Choice=   4
```

b. Find the sum of two polynomials POLY1(x,y,z) and POLY2(x,y,z) and store the result in POLYSUM(x,y,z)

OBJECTIVES:

To perform addition of two given polynomial of three variables using singly circular linked list.

REAL TIME APPLICATIONS:

- A ludo game has to come back give turn to first player after 4th player so it has to be circular.
- In our keyboard we have alt+tab key this is circular linked list it goes back on to first tab after last tab and we can go back also.

Program 9b:

```
#include<stdio.h>
#include<stdlib.h>
#include<math.h>
struct node      //Defining Polynomial fields
{
int coef, px, py, pz,flag;
struct node *link;
};
typedef struct node * NODE;
insert(NODE head,int cof,int x,int y, int z) //inserting term to poly
{
        NODE cur,tmp;
        tmp= (NODE)malloc(sizeof(struct node));    //Allocates memory
        int cf,px,py,pz;
        cur=head->link;
        tmp->coef=cof;
        tmp->px=x;
        tmp->py=y;
        tmp->pz=z;
        tmp->flag=0;
        while(cur->link!=head)          //Identifying last node
                cur=cur->link;
                cur->link=tmp;
                tmp->link=head;
}
NODE create_list(NODE head)         //For creating poly1 & poly2
{
        int i,n,cf,px,py,pz;
        printf("Enter the number of terms : ");
        scanf("%d",&n);
        for(i=1;i<=n;i++)
        {
                printf("Enter the Co-ef, px, py, pz : ");
                scanf("%d %d %d %d",&cf,&px,&py,&pz);
                insert(head,cf,px,py,pz);
```

```
        }
        return head;
}/*End of create_list()*/
NODE add_poly(NODE h1,NODE h2,NODE h3)
{
        NODE cur1,cur2;
        int scf;
        cur1=h1->link;
        cur2=h2->link;
        while(cur1 != h1)       //Till end of poly1
        {
                if(cur2 == h2)
                        cur2=h2->link;
                while(cur2 != h2)       //Till end of poly2
                {
                if(cur1->px == cur2->px && cur1->py == cur2->py && cur1->pz == cur2->pz)
                {       //Add & insert if co-ef's of both poly is equal
                                scf = cur1->coef + cur2->coef;
                                insert(h3,scf,cur1->px,cur1->py,cur1->pz);
                                cur2->flag=1;
                                cur2=h2->link;
                                break;
                }
                cur2=cur2->link;
                }
                if(cur1 == h1)
                        break;
                if(cur2 == h2) //If co-ef of poly1 is not matched, insert it to poly3
                insert(h3,cur1->coef,cur1->px,cur1->py,cur1->pz);
                cur1=cur1->link;
        }
        cur2=h2->link;
        while(cur2 != h2)       //remaining poly2 nodes inserted to poly3
        {
                if(cur2->flag==0)
                        insert(h3,cur2->coef,cur2->px,cur2->py,cur2->pz);
                cur2=cur2->link;
        }
        return h3;
}
void display(NODE head)
{
   NODE cur;
        if(head->link==head) //if poly is empty
        {
                printf("List is empty\n");
                return;
        }
        cur=head->link;
```

```
        while(cur != head)      //display all terms till end
        {
                if(cur->coef > 0)
                        printf(" +%dx^%dy^%dz^%d ",cur->coef,cur->px,cur->py,cur->pz);
                else if (cur->coef < 0)
                        printf(" %dx^%dy^%dz^%d ",cur->coef,cur->px,cur->py,cur->pz);
                        cur=cur->link;
   }
printf("\n");

}/*End of display() */

void main()
{
        int choice,data,item,pos;
   NODE head1,head2,head3;

        head1=(NODE)malloc(sizeof(struct node));
        head1->link=head1;    //poly1

        head2=(NODE)malloc(sizeof(struct node));
        head2->link=head2;    //poly2

        head3=(NODE)malloc(sizeof(struct node));
        head3->link=head3;    //poly3

        printf("\n1.Create Polynomial 1\n");
        head1=create_list(head1);

        printf("\n2.Create Polynomial 2\n");
        head2=create_list(head2);
        printf("\nPolynomial 1 is :");
        display(head1);

        printf("\nPolynomial 2 is :");
        display(head2);

        head3=add_poly(head1,head2,head3);          //Add both polynomials

        printf("\nAddition of two Polynomial is :");
        display(head3);
}
```

EXPECTED OUTPUT:

```
admin1@global:~$ ./a.out

1.Create Polynomial 1
Enter the number of terms : 2
Enter the Co-ef, px, py, pz : 2
3
4
3
Enter the Co-ef, px, py, pz : 1
3
4
3

2.Create Polynomial 2
Enter the number of terms : 2
Enter the Co-ef, px, py, pz : 3
1
2
3
Enter the Co-ef, px, py, pz : 4
2
3
1

Polynomial 1 is : +2x^3y^4z^3  +1x^3y^4z^3

Polynomial 2 is : +3x^1y^2z^3  +4x^2y^3z^1

Addition of two Polynomial is : +2x^3y^4z^3  +1x^3y^4z^3  +3x^1y^2z^3  +4x^2y^3z^1
```

PROGRAM 10

Design, Develop and Implement a menu driven Program in C for the following operations on Binary Search Tree (BST) of Integers

a. **Create a BST of N Integers: 6, 9, 5, 2, 8, 15, 24, 14, 7, 8, 5, 2**
b. **Traverse the BST in In-order, Preorder and Post Order**
c. **Search the BST for a given element (KEY) and report the appropriate message**
d. **Exit**

ABOUT THE EXPERIMENT:

Binary Tree

A tree is a acyclic, connected graph defined as a nonempty finite set of labeled nodes such that there is only one node called the root of the tree, and the remaining nodes are partitioned into sub trees. If the tree is either empty or each of its nodes has not more than two sub trees, it is called a ***binary tree***. Hence each node in a binary tree has either no children, one left child, one right child, or a left child and a right child, each child being the root of a binary tree called a sub tree.

Every node in a binary tree contains information divided into three parts. They are key field, a left-child field, and a right-child field. Whenever a node does not have a right child or a left child, then the corresponding field is set to NULL.

Binary Search Tree

A binary search tree (BST), also known as an ordered binary tree, is a node-based data structure in which each node has no more than two child nodes. Each child must either be a leaf node or the root of another binary search tree. The left sub-tree contains only nodes with keys less than the parent node; the right sub-tree contains only nodes with keys greater than the parent node.

Tree Traversal:

A tree traversal is a specific order in which to trace the nodes of a tree. In a traversal of a tree, each element of the tree is visited exactly once.

There are three tree traversal methods:

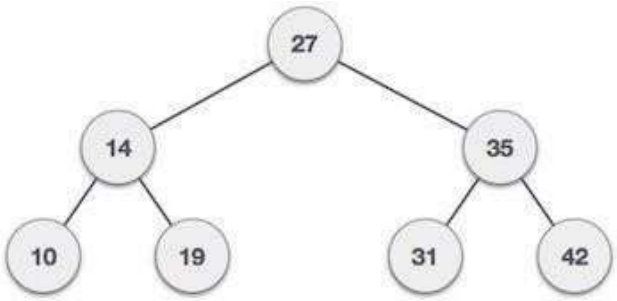
- preorder,
- inorder,
- postorder.

A **binary search tree** (BST) is a **tree** in which all nodes follows the below mentioned properties

- The left sub-**tree** of a node has key less than or equal to its parent node'skey.
- The right sub-**tree** of a node has key greater than or equal to its parent node'skey.

Thus, a binary search tree (BST) divides all its sub-trees into two segments; *left* sub-treeand *right* sub-tree and can be defined as

```
left_subtree (keys) ≤ node (key) ≤ right_subtree (keys)
```



Following are basic primary operations of a tree which are following.

- Search − search an element in a tree.
- Insert − insert an element in atree.
- Preorder Traversal − traverse a tree in a preorder manner.
- Inorder Traversal − traverse a tree in an inorder manner.
- Postorder Traversal − traverse a tree in a postorder manner.

```
inorder(root )
        if root != NULL then
                Traverse the left subtree in inorder
                Visit the root
                Traverse in right subtree in inorder
        end if
preorder( root )
        if root != NULL then
                Visit the root
                Traverse the left subtree in preorder
                Traverse in right subtree in preorder.
        end if
postorder( root )
        if root != NULL then
                Traverse the left subtree in postorder
                Traverse in right subtree in postorder.
                Visit the root.
```

OBJECTIVES:

To create a BST, traverse the BST in In-order, Preorder and Post Order and perform Search operation and report the appropriate message

REAL TIME APPLICATIONS:

- used in Unix kernels for managing a set of virtual memory areas (VMAs). Each VMA represents a section of memory in a Unix process. VMAs vary in size from 4KB to 1GB. Also want to support range queries to determine which VMAs overlap a given range used to efficiently store data in sorted form in order to access and search stored elements quickly.
- implementing routing table in router.
- used to represent arithmetic expressions

- used to implement dictionary.
- used to implement multilevel indexing in DATABASE.
- used to implement data compression code

Source code:

```
#include<stdio.h>
#include<stdlib.h>
int choice,data,key;
struct node
{
int info;
struct node *lchild,*rchild;
};
typedef struct node *NODE;

int main()
{
NODE root=NULL;
NODE CREATE(NODE,int);
void INORDER(NODE),POSTORDER(NODE),PREORDER(NODE);
NODE SEARCH_NODE(NODE,int);

while(1)
{
printf("\n1:CREATE\n2:TREE TRAVERSAL\n3.SEARCH\n4.EXIT");
printf("\nEnter your choice\n");
scanf("%d",&choice);
switch(choice)
{
case 1: printf("\nEnter data to be inserted\n");
        scanf("%d",&data);
        root=CREATE(root,data);
        break;

case 2: if(root==NULL)
            printf("\nEMPTY TREE\n");
        else
           {
            printf("\nThe Inorder display : ");
            INORDER(root);
            printf("\nThe Preorder display : ");
            PREORDER(root);
            printf("\nThe Postorder display : ");
            POSTORDER(root);
           }
        break;
   case 3: printf("\nenter the key to search:\n");
            scanf("%d",&key);
            SEARCH_NODE(root,key);
```

```
                break;

        case 4:  exit(0);
       }
   }
}

   NODE  CREATE(NODE root,int data)
   {
   NODE newnode,x,parent;
   newnode=(NODE)malloc(sizeof(struct node));
   newnode->lchild=newnode->rchild=NULL;
   newnode->info=data;

   if(root==NULL)
         root=newnode;
   else
   {
         x=root;
         while(x!=NULL)
         {
            parent=x;
            if(x->info<data)
                  x=x->rchild;
            else if(x->info>data)
                  x=x->lchild;
            else
            {
                  printf("\nNode is already present in the tree\n");
                  return(root);
            }
         }

        if(parent->info<data)
              parent->rchild=newnode;
        else
              parent->lchild=newnode;
     }

   return(root);
   }
   void INORDER(NODE root)
   {
        if(root!=NULL)
        {
             INORDER(root->lchild);
             printf("%d  ",root->info);
             INORDER(root->rchild);
        }}
```

```
void PREORDER(NODE root)
{
          if(root!=NULL)
          {
             printf("%d ",root->info);
             PREORDER(root->lchild);
             PREORDER(root->rchild);
          }
}
void POSTORDER(NODE root)
{
         if(root!=NULL)
         {
            POSTORDER(root->lchild);
            POSTORDER(root->rchild);
            printf("%d ",root->info);
         }
}

NODE SEARCH_NODE(NODE root, int key)
{
  NODE cur,q,parent,successor;
  if(root==NULL)
  {
        printf("\nTree is empty\n");
        return root;
  }
parent=NULL,cur=root;
while(cur!=NULL)
{
       if(key==cur->info)
       break;
parent=cur;
cur= (key<cur->info)?cur->lchild:cur->rchild;
}
if(cur==NULL)
{
      printf("\nData is not found\n");
      return root;
}
printf("\nData %d is found\n",key);
}
```

EXPECTED OUTPUT

```
1:CREATE
2:TREE TRAVERSAL
3.SEARCH
4.EXIT
Enter your choice
2

EMPTY TREE

1:CREATE
2:TREE TRAVERSAL
3.SEARCH
4.EXIT
Enter your choice
1

Enter data to be inserted
6

1:CREATE
2:TREE TRAVERSAL
3.SEARCH
4.EXIT
Enter your choice
1

Enter data to be inserted
9

1:CREATE
2:TREE TRAVERSAL
3.SEARCH
4.EXIT
Enter your choice
1

Enter data to be inserted
5

1:CREATE
2:TREE TRAVERSAL
3.SEARCH
4.EXIT
Enter your choice
1

Enter data to be inserted
2

1:CREATE
2:TREE TRAVERSAL
3.SEARCH
4.EXIT
Enter your choice
1

Enter data to be inserted
8
```

```
1:CREATE
2:TREE TRAVERSAL
3.SEARCH
4.EXIT
Enter your choice
1

Enter data to be inserted
15

1:CREATE
2:TREE TRAVERSAL
3.SEARCH
4.EXIT
Enter your choice
1

Enter data to be inserted
24

1:CREATE
2:TREE TRAVERSAL
3.SEARCH
4.EXIT
Enter your choice
1

Enter data to be inserted
14

1:CREATE
2:TREE TRAVERSAL
3.SEARCH
4.EXIT
Enter your choice
1

Enter data to be inserted
7

1:CREATE
2:TREE TRAVERSAL
3.SEARCH
4.EXIT
Enter your choice
1

Enter data to be inserted
8

Node is already present in the tree
```

```
1:CREATE
2:TREE TRAVERSAL
3.SEARCH
4.EXIT
Enter your choice
1

Enter data to be inserted
5

Node is already present in the tree

1:CREATE
2:TREE TRAVERSAL
3.SEARCH
4.EXIT
Enter your choice
1

Enter data to be inserted
2

Node is already present in the tree

1:CREATE
2:TREE TRAVERSAL
3.SEARCH
4.EXIT
Enter your choice
2

The Inorder display : 2 5 6 7 8 9 14 15 24
The Preorder display : 6 5 2 9 8 7 15 14 24
The Postorder display : 2 5 7 8 14 24 15 9 6
1:CREATE
2:TREE TRAVERSAL
3.SEARCH
4.EXIT
Enter your choice
3

enter the key to search:
14

Data 14 is found

1:CREATE
2:TREE TRAVERSAL
3.SEARCH
4.EXIT
Enter your choice
3

enter the key to search:
4

Data is not found
```

PROGRAM 11

Design, develop and implement a Program in C for the following operations on Graph (G) of Cities

a. **Create a Graph of N cities using Adjacency Matrix.**
b. **Print all the nodes reachable from a given starting node in a digraph using BFS method**

ABOUT THE EXPERIMENT:

Breadth-first search (BFS):

Breadth-first search (BFS) is an algorithm for traversing or searching tree or graph data structures. It starts at the tree root and explores the neighbor nodes first, before moving to the next level neighbors. Breadth First Search algorithm(BFS) traverses a graph in a breadth wards motion and uses a queue to remember to get the next vertex to start a search when a dead end occurs in any iteration.

Depth-first search (DFS):

Depth-first search (DFS) is an algorithm for traversing or searching tree or graph data structures. One starts at the root (selecting some arbitrary node as the root in the case of a graph) and explores as far as possible along each branch before backtracking.

Depth-first search, or DFS, is a way to traverse the graph. Initially it allows visiting vertices of the graph only, but there are hundreds of algorithms for graphs, which are based on DFS. Therefore, understanding the principles of depth-first search is quite important to move ahead into the graph theory. The principle of the algorithm is quite simple: to go forward (in depth) while there is such possibility, otherwise tobacktrack.

Adjacency Matrix

In **graph** theory, computer science, an **adjacency matrix** is a square **matrix** used to **represent** a finite **graph**. The elements of the **matrix** indicate whether pairs of vertices are adjacent or not in the **graph**. In the special case of a finite simple **graph**, the **adjacency matrix** is a (0, 1)- **matrix** with zeros on its diagonal.

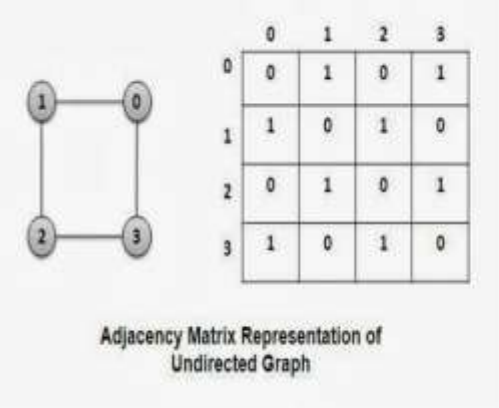
A graph G = (V, E) where v= {0, 1, 2, . . .n-1} can be represented using two dimensional integer array of size n x n. a[20][20] can be used to store a graph with 20 vertices.

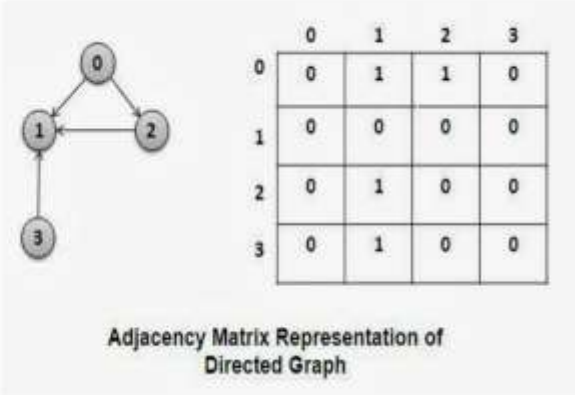
a[i][j] = 1, indicates presence of edge between two vertices i and j.

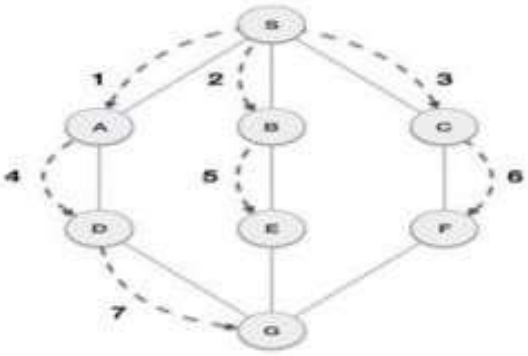
a[i][j] = 0, indicates absence of edge between two vertices i and j.

- A graph is represented using squarematrix.
- Adjacency matrix of an undirected graph is always a symmetric matrix, i.e. an edge (i, j) implies the edge (j,i).
- Adjacency matrix of a directed graph is never symmetric, adj[i][j] = 1 indicates a directed edge from vertex i to vertexj.

An example of adjacency matrix representation of an undirected and directed graph is given below:







As in example given above, BFS algorithm traverses from A to B to E to F first then to C and G lastly to D. It employs following rules.

- **Rule 1** − Visit adjacent unvisited vertex. Mark it visited. Display it. Insert it in aqueue.
- **Rule 2** − If no adjacent vertex found, remove the first vertex from queue.
- **Rule 3** − Repeat Rule 1 and Rule 2 until queue isempty.

OBJECTIVES:

a. Create a Graph of N cities using given Adjacency Matrix for a given graph
b. using BFS method, print all the nodes reachable from a given starting node in a digraph

REAL TIME APPLICATIONS:

- In **Computer science** graphs are used to represent the flow of computation.
- **Google maps** uses graphs for building transportation systems, where intersection of two(ormore) roads are considered to be a vertex and the road connecting two vertices is considered tobe an edge, thus their navigation system is based on the algorithm to calculate the shortest path between two vertices.
- In **Facebook**, users are considered to be the vertices and if they are friends then there is an edge running between them. Facebook's Friend suggestion algorithm uses graph theory. Facebook is an example of **undirected graph**.

- In **World Wide Web**, web pages are considered to be the vertices. There is an edge from a page u to other page v if there is a link of page v on page u. This is an example of **Directed graph**. It was the basic idea behind Google Page Ranking Algorithm.
- In **Operating System**, we come across the Resource Allocation Graph where each process and resources are considered to be vertices. Edges are drawn from resources to the allocated process, or from requesting process to the requested resource. If this leads to any formation of a cycle then a deadlock will occur.

Source Code:

```
#include<stdio.h>
#include<stdlib.h>
int n,a[10][10],i,j,source,s[10],choice,count;
void bfs(int n,int a[10][10],int source,int s[]) //BFS Algorithm
{
        int q[10],u;
        int front=1,rear=1;
        s[source]=1;
        q[rear]=source;
        while(front<=rear)
        {
                u=q[front];
                front=front+1;
                for(i=1;i<=n;i++)
                if(a[u][i]==1 &&s[i]==0)
                {
                        rear=rear+1;
                        q[rear]=i;
                        s[i]=1;
                }
        }
}
int main()
{
        printf("Enter the number of nodes : ");
        scanf("%d",&n);
        printf("\n Enter the adjacency matrix\n");
        for(i=1;i<=n;i++)       //Provide matrix of 0's and 1's
                for(j=1;j<=n;j++)
                        scanf("%d",&a[i][j]);
        while(1)
        {
                printf("\nEnter your choice\n");
                printf("1.BFS\n 2.Exit\n");
                scanf("%d",&choice);
                switch(choice)
                {
                        case 1: printf("\n Enter the source :");
```

```
                        scanf("%d",&source); //Provide source for BFS
                        for(i=1;i<=n;i++)
                        s[i]=0;
                        bfs(n,a,source,s);
                        for(i=1;i<=n;i++)
                        {
                                if(s[i]==0)
                                        printf("\n The node %d is not reachable",i);
                                else
                                        printf("\n The node %d is reachable",i);
                        }
                        break;
                case 2: exit(0);
            }
        }
}
```

EXPECTED OUTPUT:

```
Enter the number of nodes : 5

 Enter the adjacency matrix
0 1 0 1 0
0 0 1 0 0
1 0 0 0 0
0 0 1 0 0
0 0 1 1 0

Enter your choice
1.BFS
 2.Exit
1

 Enter the source :5

 The node 1 is reachable
 The node 2 is reachable
 The node 3 is reachable
 The node 4 is reachable
 The node 5 is reachable
Enter your choice
1.BFS
 2.Exit
1

 Enter the source :1

 The node 1 is reachable
 The node 2 is reachable
 The node 3 is reachable
 The node 4 is reachable
 The node 5 is not reachable
Enter your choice
1.BFS
 2.Exit
```

//Print all the nodes reachable from a given starting node in a digraph using DFS method

Source Code:

```
#include<stdio.h>
 void dfs(int n,int a[10][10],int source,int s[])
 {
   int i;
   s[source]=1;
    for(i=1;i<=n;i++)
        if(a[source][i]==1 && s[i]==0)
            dfs(n,a,i,s);
 }
 int main()
 {
          int i,j,source,s[10];
          int n,a[10][10];
          int count=0;
          printf("\nEnter the number of nodes : ");
          scanf("%d",&n);
          printf("\nEnter the paths \n");
          for(i=1;i<=n;i++)
             for(j=1;j<=n;j++)
                 scanf("%d",&a[i][j]);

          printf("\nEnter the source vertex :");
          scanf("%d",&source);
          for(i=1;i<=n;i++)
                   s[i]=0;
          dfs(n,a,source,s);
           for(i=1;i<=n;i++)
            {
              if(s[i]==0)
                   printf("\n The node %d is not reachable",i);
              else
                   printf("\n The node %d is reachable",i);
            }
    }
```

PROGRAM 12

Given a File of N employee records with a set K of Keys(4-digit) which uniquely determine the records in file F. Assume that file F is maintained in memory by a Hash Table(HT) of mmemory locations with L as the set of memory addresses (2-digit) of locations in HT. Let the keys in K and addresses in L are Integers.

Design and develop a Program in C that uses Hash function H: K->L as H(K)=K mod m (remainder method), and implement hashing technique to map a given key K to the address space L. Resolve the collision (if any) using linear probing

ABOUT THE EXPERIMENT:

Hash table is one of the classic data structures of computer science. It provides constant-time access to key/value data in the average case. The idea is to store a key/value pair in a location that is computed based on the value of the key, which works fine when all the hash values are unique. The problem arises when two keys hash to the same value called *collision.*

Open addressing computes a second address, or if necessary a third, or fourth, or so on, continuing until it finds an empty spot. It is necessary that the computation of secondary addresses eventually visits every possible storage location. *Linear probing is a simple approach i*n which storage locations are accessed in increasing order until an empty location is found. It is possible, of course, for the hash table to become completely filled, in which case an error is reported when a new item cannot be inserted.

S.n.	Key	Hash	Array Index	After Linear Probing, ArrayIndex
1	1	1 % 20 = 1	1	1
2	2	2 % 20 = 2	2	2
3	42	42 % 20 = 2	2	3
4	4	4 % 20 = 4	4	4
5	12	12 % 20 = 12	12	12
6	14	14 % 20 = 14	14	14
7	17	17 % 20 = 17	17	17
8	13	13 % 20 = 13	13	13
9	37	37 % 20 = 17	17	18

Following are operations of a hashtable are performed:

- create − create a key for given element .
- Insert − insert an element in a hashtable.
- display − display all elements from a hashtable.

OBJECTIVES:

use hash function H(K)=K mod m (remainder method) to implement hashing technique to map a given key K(4-digit) to the address space L(2-digit) using linear probing technique to resolve collisions.

REAL TIME APPLICATIONS:

Password Verification:

Cryptographic hash functions are very commonly used in password verification. Let's understand this using an Example: When you use any online website which requires a user login, you enter your E-mail and password to authenticate that the account you are trying to use belongs to you. When the password is entered, a hash of the password is computed which is then sent to the server for verification of the password. The passwords stored on the server are actually computed hash values of the original passwords. This is done to ensure that when the password is sent from client to server, no sniffing is there.

Rabin-Karp Algorithm:

One of the most famous applications of hashing is the Rabin-Karp algorithm. This is basically a string-searching algorithm which uses hashing to find any one set of patterns in a string. A practical application of this algorithm is detecting plagiarism

Linking File name and path together:

When moving through files on our local system, we observe two very crucial components of a file i.e. file_name and file_path. In order to store the correspondence between file_name and file_path the system uses a map(file_name, file_path)which is implemented using a hash table.

Compiler Operation:

The keywords of a programming language are processed differently than other identifiers. To differentiate between the keywords of a programming language(if, else, for, return etc.) and other identifiers and to successfully compile the program, the compiler stores all these keywords in a set which is implemented using a hash table.

Source Code:

```
#include<stdio.h>
#include<stdlib.h>
#define MAX 100
int create(int);
void linearprobe(int[], int, int);
void Display(int []);
int count=0;
int  main()
{
        int a[MAX], num, key,i;
        int ans=1;
        printf("collision handling by linear probing\n");
        for(i=0;i<MAX;i++)
           a[i]= -1;
        do
        {
                printf("enter the data\n");
                scanf("%4d", &num);
                key= create(num);
```

```
                linearprobe(a,key,num);
                printf(" do you wish to continue?(1/0)\n");
                scanf("%d", &ans);
        }
        while(ans);
        Display(a);
        return 0;
}
int create(int num)
{
        int key;
        key=num%100;
        return key;
}
void linearprobe(int a[MAX], int key, int num)
{
        int index=0, i;
        if(a[key]==-1)
        {
                a[key]=num;
                count++;
        }
        else
        {
                printf("collision detected!\n");
                if(count==MAX)
                {
                        printf("hash table is full\n");
                        Display(a);
                        exit(1);
                 }
                 printf("coliision avoided successfully using LINEAR PROBING\n");
                 for(i=1;i<MAX;i++)
                 {
                        index=(key+i)%MAX;
                        if(a[index]==-1)
                                {
                                  a[index]=num;
                                  count++;
                                  break;
                                }
                 }

          }
}
void Display(int a[MAX])
{
        int i, choice;
        printf("\n1.display All \t 2. Filtered Display\n");
```

```
        scanf("%d", &choice);
        printf("\n total no of record present in hash: %d ",count);
        if(choice==1)
         {
              printf("\n hash table is\n");
              for(i=0;i<MAX;i++)
                    printf("\n%d\t%d",i,a[i]);
         }
        else
        {
                  printf("hash table is\n");
                  for(i=0;i<MAX;i++)
                      if(a[i]!=-1)
                     {
                           printf("\n%d\t%d\n",i,a[i]);
                           continue;
                     }
         }
}
```

EXPECTED OUTPUT:

```
enter the data
1441
 do you wish to continue?(1/0)
1
enter the data
2141
collision detected!
coliision avoided successfully using LINEAR PROBING
 do you wish to continue?(1/0)
1
enter the data
1000
 do you wish to continue?(1/0)
1
enter the data
1
 do you wish to continue?(1/0)
1
enter the data
1001
collision detected!
coliision avoided successfully using LINEAR PROBING
 do you wish to continue?(1/0)
1
enter the data
1999
 do you wish to continue?(1/0)
1
enter the data
3999
collision detected!
coliision avoided successfully using LINEAR PROBING
 do you wish to continue?(1/0)
0

1.display All    2. Filtered Display
2

 total no of record present in hash: 7 hash table is
```

```
0       1000

1       1

2       1001

3       3999

41      1441

42      2141

99      1999
admln1@admln1-HP-ProDesk-400-G3-DM:~/Desktop$
```

CHAPTER 3

ADDITIONAL PROGRAMS

1. Write a 'C' program to determine if a given matrix is a Sparse matrix or not

```
#include<stdio.h>
void main()
{
        staticint array[10][10];
        inti, j, m, n;
        int counter = 0;
        printf("Enter the order of the matix \n");
        scanf("%d %d", &m, &n);

        printf("Enter the co-efficients of the matix \n");
        for(i = 0; i< m; ++i)
        {
        for(j = 0; j < n; ++j)
         {
                        scanf("%d",&array[i][j]);
                        if(array[i][j] == 0)
                {
                                ++counter;
                 }
               }
        }
        if(counter > ((m * n) / 2))
        {
        printf("The given matrix is sparse matrix \n");
        }
        else
        printf("The given matrix is not a sparse matrix \n");
        printf("There are %d number of zeros", counter);
}
```

OUTPUT

$ cc pgm_sparse.c
$ a.out
Enter the order of the matix
3 3
Enter the co-efficients of the matix
10 20 30
5 10 15
3 6 9
The given matrix is not a sparse matrix
There are 0 number of zeros
	$ a.out
	Enter the order of the matrix
	3 3
	Enter the co-efficients of the matrix
	5 0 0
	0 0 5
	0 5 0
	The given matrix is sparse matrix
	There are 6 number of zeros

2. Write a C program to demonstrate Ackermann's Function

```
#include<stdio.h>
#include<stdlib.h>
intackermann(int x, int y);
int count = 0, indent = 0;
void main(intargc, char **argv)
{
        intx,y;
        if(argc!=3)
        {
                printf("\nUsage : %s <number1><number2>\n",argv[0]);
                exit(0);
        }
        x=atoi(argv[1]);
        y=atoi(argv[2]);
        printf("\nAckermann Function with inputs (%d,%d) is %d\n",x,y,ackermann(x,y));
        printf("\nFunction called %d times.\n",count);
}
intackermann(int x, int y)
{
        count++;
        if(x<0 || y<0)
                return -1;
        if(x==0)
                return y+1;
        if(y==0)
                returnackermann(x-1,1);
        returnackermann(x-1,ackermann(x,y-1));
}
```

INPUT/ OUTPUT:

```
$cc acker.c
$./a.out 2 3
Ackermann Function with inputs (2,3) is 9
Function called  44  times.
```

3.Write a C program to find the largest element in a given doubly linked list.

```
#include  <stdio.h>
#include <stdlib.h>
struct node
{
   intnum;
   struct node *next;
   struct node *prev;
};
void create(struct node **);
int max(struct node *);
void release(struct node **);
int main()
{
   struct node *p = NULL;
   int n;
   printf("Enter data into the list\n");
   create(&p);
   n = max(p);
   printf("The maximum number entered in the list is %d.\n", n);
   release (&p);
   return 0;
}
int max(struct node *head)
{
   struct node *max, *q;
   q = max = head;
   while (q != NULL)
   {
      if (q->num> max->num)
      {
         max = q;
      }
      q = q->next;
   }
   return (max->num);
}
void create(struct node **head)
{
   int c, ch;
   struct node *temp, *rear;
   do
   {
      printf("Enter number: ");
      scanf("%d", &c);

      temp = (struct node *)malloc(sizeof(struct node));
      temp->num = c;
```

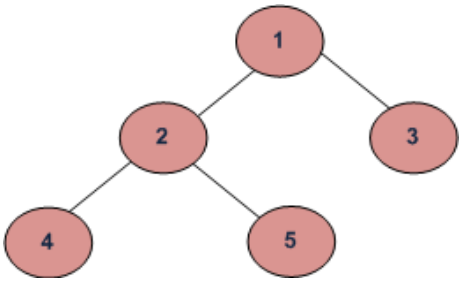
```
        temp->next = NULL;
        temp->prev = NULL;
        if (*head == NULL)
        {
           *head = temp;
        }
        else
        {
           rear->next = temp;
           temp->prev = rear;
        }
        rear = temp;

        printf("Do you wish to continue [1/0]: ");
        scanf("%d", &ch);
     }
     while (ch !=0);
     printf("\n");
}
void release(struct node **head)
{
     struct node *temp = *head;
     *head = (*head)->next;
     while ((*head) != NULL)
     {
        free(temp);
        temp = *head;
        (*head) = (*head)->next;
     }
}
```

INPUT/ OUTPUT:

```
Enter data into the list
Enter number: 12
Do you wish to continue [1/0]: 1
Enter number: 7
Do you wish to continue [1/0]: 1
Enter number: 23
Do you wish to continue [1/0]: 1
Enter number: 4
Do you wish to continue [1/0]: 1
Enter number: 1
Do you wish to continue [1/0]: 1
Enter number: 16
Do you wish to continue [1/0]: 0
The maximum number entered in the list is 23
```

4.Write a C program to find the maximum depth or height of a tree

Given a binary tree, find height of it. Height of empty tree is 0 and height of below tree is 3.




```
#include<stdio.h>
#include<stdlib.h>
/* A binary tree node has data, pointer to left child    and a pointer to right child */
struct node
{
int data;
struct node* left;
struct node* right;
};
/* Compute the "maxDepth" of a tree -- the number of nodes along the longest path from the root node
down to the farthest leaf node.*/
intmaxDepth(struct node* node)
{
if (node==NULL)
return 0;
else
  {
     /* compute the depth of each subtree */
intlDepth = maxDepth(node->left);
intrDepth = maxDepth(node->right);
     /* use the larger one */
if (lDepth>rDepth)
return(lDepth+1);
else return(rDepth+1);
  }
}
/* Helper function that allocates a new node with the given data and NULL left and right pointers. */
struct node* newNode(int data)
{
struct node* node = (struct node*)malloc(sizeof(struct node));
node->data = data;
node->left = NULL;
node->right = NULL;
return(node);
}
void main()
{
struct node *root = newNode(1);
```

```
root->left = newNode(2);
root->right = newNode(3);
root->left->left = newNode(4);
root->left->right = newNode(5);
printf("Height of tree is %d\n", maxDepth(root));
}
```

INPUT/ OUTPUT:

```
Height of tree is 3
//add root->left->right->right= newNode(9);
before printf() in main()
Output:
Height of tree is 4
```

5.Write a C program to arrange a list of integers in ascending order using Insertion sort

```
#include<stdio.h>
voidinst_sort(intnum[]);
void main()
{
        intnum[5],count;

        printf("\n enter the five elements to sort:\n");
        for(count=0;count<5;count++)
                scanf("%d",&num[count]);

        inst_sort(num); /*function call for insertion sort*/

        printf("\n\n elements after sorting:\n");
        for(count=0;count<5;count++)
                printf("%d\n",num[count]);
}
voidinst_sort(intnum[])
{
        inti,j,k;
        for(j=1;j<5;j++)
        {
                k=num[j];
                for(i=j-1;i>=0&&k<num[i];i--)
                        num[i+1]=num[i];
                num[i+1]=k;
        }
}
```

INPUT/ OUTPUT:

Enter the five elements to sort: 3 2 1 2 7	Elements after sorting: 1 2 2 3 7

CHAPTER 4

Sample Viva Questions with Answers

1. **What is data structure?**
   A data structure is a way of organizing data that considers not only the items stored, but also their relationship to each other.
2. **List out the areas in which data structures are applied extensively?**
   - Compiler Design
   - Operating System
   - Database Management System
   - Statistical analysis package
   - Numerical Analysis
   - Graphics
   - Artificial Intelligence
   - Simulation
3. **What are the major data structures used in the following areas: RDBMS, Network data model and Hierarchical data model?**
   - RDBMS = Array (i.e. Array of structures)
   - Network data model = Graph
   - Hierarchical data model = Trees
4. **What is the Minimum number of queues needed to implement the priority queue?**
   Two. One queue is used for actual storing of data and another for storing priorities.
5. **What is the data structures used to perform recursion?**
   Stack. Because of its LIFO (Last In First Out) property it remembers its 'caller' so knows whom to return when the function has to return.
6. **In tree construction which is the suitable efficient data structure? (Array, Linked list, Stack, Queue)**
   Linked list is the suitable efficient data structure.
7. **What is the significant difference between ARRAY and STACK?**
   Stack uses LIFO. Thus the item that is certainly first entered would be the last removed. In array the items could be entered or removed in any order. Basically each and every member access is done making use of index and no strict order is to be followed here to remove a particular element. Array could be multi-dimensional or one dimensional but stack really should be one- dimensional. Size of array is fixed, while stack may be grow or shrink. We can say stack is actually dynamic data structure.
8. **Explain whether Linked List is actually linear or Non-linear data structure?** Link list is definitely obviously linear data structure simply because each and every element (NODE) acquiring specific place and as well each and every component has got its unique successor in addition to predecessor.
9. **What is the difference between a Stack and a Queue?**
   Stack follows LIFO (Last In First Out) method of accessing the members.
   Queue follows FIFO (First In First Out) method of accessing the members.
10. **What is a one way chain or singly linked linear list?**
    Singly linked list is a collection of nodes. Each node has element and address of the nextelement. With that address only forward traversal is possible i.e. single link

11. How can a node be inserted in the middle of a linked list?

By repointing the previous and the next elements of existing nodes to the new node.

12. What is the heap?

The heap is where malloc(), calloc(), and realloc() get memory.

13. Which data structure is needed to convert infix notations to postfix notations?

Stack

14. List out few of the applications that make use of Multilinked Structures?

- Sparse matrix.
- Index generation.

15. In tree construction which is the suitable efficient data structure?

Linked List

16. What is the evaluation order according to which an infix expression is converted to postfix expression?

The evaluation order is:

- Brackets or Parenthesis,
- Exponentiation,
- Multiplication or division,
- Addition or subtraction.

17. Give the advantages of using postfix notations over infix notations?

An infix expression is evaluated keeping in mind the precedence of operators. An infix expression is difficult for the machine to know and keep track of precedence of operators. A postfix expression itself takes care of the precedence of operators as the placement of operators. Thus, for the machine, it is easier to carry out a postfix expression than an infix expression.

18. List out few of the Application of tree data-structure?

The manipulation of Arithmetic expression, Symbol Table construction, Syntax analysis.

19. Why do we Use a Multidimensional Array?

A multidimensional array can be useful to organize subgroups of data within an array.

20. Run Time Memory Allocation is known as?

Allocating memory at runtime is called dynamically allocating memory. In this means youdynamically allocate memory by using malloc.

21. What are the different types of queues available?

Linear Queue, Circular Queue, Priority Queue

22. What are the applications of Queue?

Round robin technique for processor scheduling, all types of customer services like railwayticket reservation center, printer server routines can be implemented using queue.

23. What is meant by top of stack?

Top is a pointer pointing to the top of the stack. Insertion and deletion can be done only at the topof the stack.

24. What is the benefit of using stacks?

Stacks provide a very efficient storage structure for manipulating data that is accessed. accordingto the LIFO method. Stacks are often used as temporarystorage were the elements stored last needs to be removed first.

Appendix

How to Compile and Run a C Program on Ubuntu Linux

Step 1. Open up a terminal Search for the terminal application in the Dash tool (located as the topmost item in the Launcher). Open up a terminal by clicking on the icon.

Step 2. Use a text editor to create the C source code.

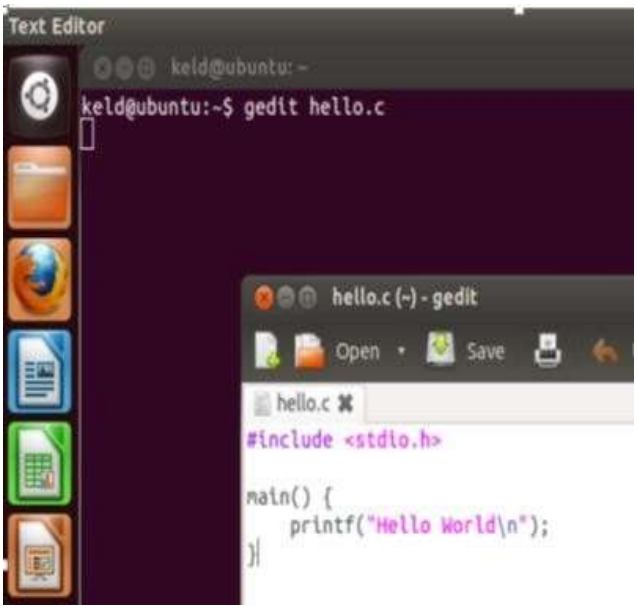
Type the command

gedithello.c

and enter the C source code below:

```
#include <stdio.h>
void main() {
        printf("Hello World\n");
}
```

Close the editor window



Step 3. Compile the program.

Type the command

gcchello.c

or

cc hello.c

This command will invoke the GNU C compiler to compile the file hello.c

Step 4. Execute the program.

Type the command

./a.out

This should result in the output

Hello World

Gedit

Gedit, the default GUI editor if you use Gnome, also runs under KDE and other desktops. Most gNewSense and Ubuntu installations use Gnome by default. To start Gedit open a terminal and type

$gedit

You should see this:

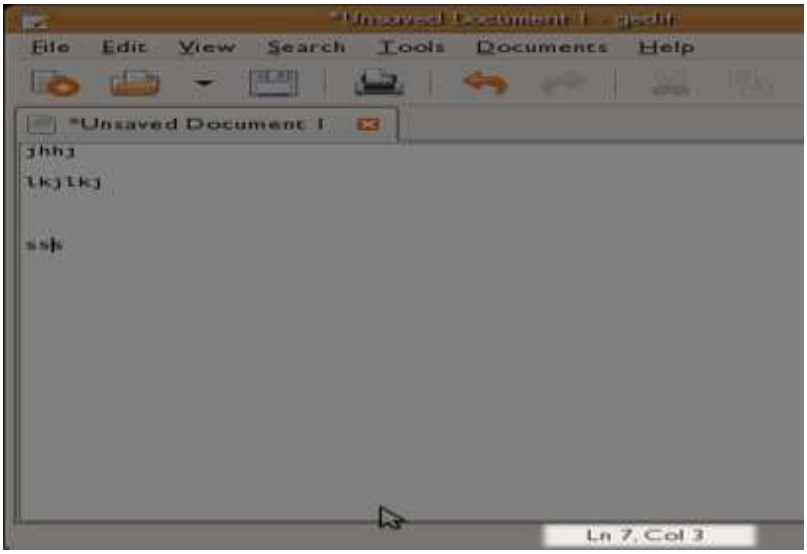
This looks like most basic editors on any operating system. You can use Gedit through the GUI, and the commands are simple:

File->Open : Opens an existing file
File->New : Creates a new (blank) file
File->Save : Saves a file
ctrl + c : copy
ctrl + v : paste

That's all you really need to do. To add text just type!

Line Numbers

Gedit tracks your cursor and displays the position at the bottom of the interface:



This can be handy information to know. If you keep track of the line numbers you can use these to jump quickly around the text file by using the "Go to Line" feature. This can be accessed via the interface (**Search -> Go to Line**) or via the shortcut **ctrl** + **i**.

gedit Editor

gedit commands:

Files	
Ctrl + N	Create a new document.
Ctrl + O	Open a document.
Ctrl + L	Open a location.
Ctrl + S	Save the current document to disk.
Ctrl + Shft + S	Save the current document with a new filename.
Ctrl + P	Print the current document.
Ctrl + Shft + P	Print preview.
Ctrl + W	Close the current document.
Ctrl + Q	Quit Gedit.
Edit	
Ctrl + Z	Undo the last action.
Ctrl + Shft + Z	Redo the last undone action.
Ctrl + X	Cut the selected text or region and place it on the clipboard.
Ctrl + C	Copy the selected text or region onto the clipboard.
Ctrl + V	Paste the contents of the clipboard.
Ctrl + A	Select all.
Ctrl + F	Find a string.
Ctrl + I	Goto line.